



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For



AI Vibe Coding with Rust

TGS Ref No: TGS-2023039924

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 3.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	Not recorded	Initial release (Methodologies in C Programming)	Tertiary Infotech Academy
2.0	13 September 2026	Rebuilt as AI-Assisted C Programming for Arduino with 14 activities	Dr Alfred Ang
3.0	27 September 2026	Course retitled AI Vibe Coding with Rust: all topics, slides, Learner Guide, Lesson Plan, 14 verified Rust activity crates and assessments rewritten	Dr Alfred Ang

TABLE OF CONTENTS

How to Use This Guide.....	6
Course Setup.....	6
Course Outcomes and Assessment.....	6
Topic 1: Rust Programming Fundamentals and Software Components (Slides 16-55).....	7
A specification becomes a component contract (Slides 17-19).....	7
The Rust toolchain: rustup, cargo and rustc (Slides 20-22).....	7
Anatomy of a Cargo package (Slides 23-25).....	8
Variables are immutable by default (Slides 26-28).....	9
Choose integer types deliberately (Slides 29-31).....	9
Functions: the signature is the contract (Slides 32-34).....	10
String and &str: owned text and borrowed text (Slides 35-37).....	10
Ownership moves; borrowing lends (Slides 38-40).....	11
Activity 1: Turn a specification into a Rust function (Slides 41-45).....	11
Step-by-step: Activity 1.....	12
Commands: Activity 1.....	12
Vibe-coding prompts: Activity 1.....	12
Reference solution: Activity 1.....	13
Acceptance tests: Activity 1.....	13
Activity 2: Validate a product code with strings (Slides 46-50).....	14
Step-by-step: Activity 2.....	14
Commands: Activity 2.....	15
Vibe-coding prompts: Activity 2.....	15
Reference solution: Activity 2.....	15
Acceptance tests: Activity 2.....	16
Activity 3: Share inventory records with ownership and borrowing (Slides 51-55).....	16
Step-by-step: Activity 3.....	17
Commands: Activity 3.....	17
Vibe-coding prompts: Activity 3.....	17
Reference solution: Activity 3.....	18
Acceptance tests: Activity 3.....	18
Topic 2: AI Vibe Coding for Rust Application Development (Slides 56-95).....	19
The vibe-coding loop for Rust (Slides 57-59).....	19
Write a bounded prompt (Slides 60-62).....	20
The compiler is your first reviewer (Slides 63-65).....	21
Review AI-generated Rust before accepting it (Slides 66-68).....	21
Modules and visibility organise a crate (Slides 69-71).....	22
Cargo dependencies and Cargo.lock (Slides 72-74).....	22
fmt and clippy encode team practice (Slides 75-77).....	23
Set up VS Code for Rust vibe coding (Slides 78-80).....	23
Activity 4: Vibe-code a command parser (Slides 81-85).....	24
Step-by-step: Activity 4.....	24
Commands: Activity 4.....	25
Vibe-coding prompts: Activity 4.....	25
Reference solution: Activity 4.....	25
Acceptance tests: Activity 4.....	26

Activity 5: Organise a crate into modules (Slides 86-90).....	27
Step-by-step: Activity 5.....	27
Commands: Activity 5.....	28
Vibe-coding prompts: Activity 5.....	28
Reference solution: Activity 5.....	29
Acceptance tests: Activity 5.....	29
Activity 6: Load inventory JSON with Cargo dependencies (Slides 91-95).....	30
Step-by-step: Activity 6.....	30
Commands: Activity 6.....	31
Vibe-coding prompts: Activity 6.....	31
Reference solution: Activity 6.....	31
Acceptance tests: Activity 6.....	32
Topic 3: Rust Programming Controls, Functions and Features (Slides 96-136).....	33
if and match are expressions (Slides 97-99).....	33
enum + match model every case (Slides 100-102).....	34
Loops: for, while and loop (Slides 103-105).....	34
Structs and impl blocks (Slides 106-108).....	35
Collections: Vec and HashMap (Slides 109-111).....	35
Traits and generics share behaviour (Slides 112-114).....	36
Closures and iterator chains (Slides 115-117).....	36
Option and Result are control features (Slides 118-120).....	37
Activity 7: Apply stock movements with enums and match (Slides 121-125).....	37
Step-by-step: Activity 7.....	38
Commands: Activity 7.....	38
Vibe-coding prompts: Activity 7.....	38
Reference solution: Activity 7.....	39
Acceptance tests: Activity 7.....	40
Activity 8: Track stock in a HashMap (Slides 126-130).....	41
Step-by-step: Activity 8.....	41
Commands: Activity 8.....	42
Vibe-coding prompts: Activity 8.....	42
Reference solution: Activity 8.....	43
Acceptance tests: Activity 8.....	43
Activity 9: Price mixed order lines with traits and iterators (Slides 131-135).....	44
Step-by-step: Activity 9.....	45
Commands: Activity 9.....	45
Vibe-coding prompts: Activity 9.....	45
Reference solution: Activity 9.....	46
Acceptance tests: Activity 9.....	47
Topic 4: Testing, Debugging and Integrating Rust Components (Slides 137-174).....	48
Unit tests live beside the code (Slides 138-140).....	48
Integration tests check components together (Slides 141-143).....	49
Custom errors and the ? operator (Slides 144-146).....	49
Read a failing test like a detective (Slides 147-149).....	50
Debug with dbg!, focused tests and AI (Slides 150-152).....	51
Interoperability: arguments, files, exit codes (Slides 153-155).....	51
Serde: typed data at the boundary (Slides 156-158).....	52

Activity 10: Parse CSV records with Result and custom errors (Slides 159-163).....	52
Step-by-step: Activity 10.....	52
Commands: Activity 10.....	53
Vibe-coding prompts: Activity 10.....	53
Reference solution: Activity 10.....	53
Acceptance tests: Activity 10.....	55
Activity 11: Debug a failing integration with tests (Slides 164-168).....	56
Step-by-step: Activity 11.....	56
Commands: Activity 11.....	57
Vibe-coding prompts: Activity 11.....	57
Reference solution: Activity 11.....	58
Acceptance tests: Activity 11.....	59
Activity 12: Integrate components into a tested command-line tool (Slides 169-173).....	60
Step-by-step: Activity 12.....	60
Commands: Activity 12.....	61
Vibe-coding prompts: Activity 12.....	61
Reference solution: Activity 12.....	61
Acceptance tests: Activity 12.....	63
Topic 5: AI-Assisted Rust Software Design and Documentation (Slides 175-207).....	64
Doc comments become documentation (Slides 176-178).....	64
Doctests keep examples honest (Slides 179-181).....	65
Trace requirements to code and tests (Slides 182-184).....	65
Architecture and module maps (Slides 185-187).....	66
Record design decisions (Slides 188-190).....	66
AI-assisted documentation, human-verified (Slides 191-193).....	67
Hand over: README and maintenance notes (Slides 194-196).....	67
Activity 13: Document an API with rustdoc and doctests (Slides 197-201).....	68
Step-by-step: Activity 13.....	68
Commands: Activity 13.....	69
Vibe-coding prompts: Activity 13.....	69
Reference solution: Activity 13.....	69
Acceptance tests: Activity 13.....	70
Activity 14: Design and document a reorder feature with AI (Slides 202-206).....	71
Step-by-step: Activity 14.....	71
Commands: Activity 14.....	72
Vibe-coding prompts: Activity 14.....	72
Reference solution: Activity 14.....	72
Acceptance tests: Activity 14.....	73
Quick Command Reference.....	75
AI Review Checklist.....	75
Support and Assessment Flow.....	75
Sources.....	76

How to Use This Guide

This Learner Guide accompanies the AI Vibe Coding with Rust course (TGS-2023039924, version 3.0). The slides teach each concept with code, a contract and a failure mode; this guide repeats that teaching and adds the complete, step-by-step instructions for all 14 hands-on activities, the reference code and the commands that prove your work.

StockPilot is the stock service of a small electronics retailer. Every activity builds or reviews one component of it: order lines, product codes, inventory records, commands, reports, data files, reorder planning and the documentation handed to the maintenance team.

Every activity folder in the course repository contains a starter/ crate for you to complete and a solution/ crate for comparison. The solution crates were compiled and tested with rustc 1.93.1 (01f6ddf75 2026-02-11) before release.

Course Setup

Complete this setup before Day 1 (about 15 minutes). You need a Windows or Mac laptop with internet access.

Step	What to do	Check
1. Install Rust	macOS/Linux: run the rustup command below. Windows: download rustup-init.exe from rustup.rs and accept the defaults (install the Visual Studio C++ build tools when prompted).	rustc --version
2. Add components	rustup component add clippy rustfmt	cargo clippy --version
3. Install VS Code	Install Visual Studio Code and the rust-analyzer extension.	Hover a variable to see its type
4. AI assistant	Use any AI coding assistant (for example GitHub Copilot Chat, Claude or ChatGPT). No API key is needed.	Assistant answers a question
5. Get the activities	git clone the course repository, or download it as a ZIP from GitHub and unzip it.	labs/ folder has 14 activities

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
rustup component add clippy rustfmt
rustc --version
cargo --version
git clone https://github.com/tertiarycourses/TGS-2023039924-AI-Vibe-Coding-with-Rust.git
cd TGS-2023039924-AI-Vibe-Coding-with-Rust/labs/activity-01-line-total/solution
cargo test
```

Course Outcomes and Assessment

LO1: Determine basic software components using programming methodologies to meet functional specifications (K1/A1).

LO2: Apply programming methodologies and tools for software creation (K2/A2).

LO3: Select essential programming controls and features to meet software design requirements (K3/A3).

LO4: Examine the interoperability and functionality of programming components (K4/A4).

LO5: Generate programming design documentation aligned with user specifications (K5/A5).

Duration: 2 days, 16 hours (14 training hours and 2 assessment hours), 9:30 AM to 6:30 PM. On Day 2 the Written Assessment (5 open-ended questions, K1-K5, 60 minutes) runs 4:30-5:30 PM and the Practical Performance (5 hands-on tasks, A1-A5, 60 minutes) runs 5:30-6:30 PM. Both are open book and individual. Download papers and upload answers on the LMS: <https://lms-tms.tertiaryinfotech.com/>

Topic 1: Rust Programming Fundamentals and Software Components (Slides 16-55)

K1/A1 / LO1: Determine basic software components using programming methodologies to meet functional specifications (K1/A1).

A specification becomes a component contract (Slides 17-19)

Requirement: Write the rule in business words before any code exists.

Input: Each input gets a type that already excludes invalid values (u32 is never negative).

Output: Option<u64> makes "refused" part of the signature, not a comment.

Evidence: Each requirement maps to at least one named test.

```
R1 price one order line in cents
R2 quantity must be 1..=10,000
R3 never wrap or lose money on overflow
```

```
component: fn line_total(
    unit_price_cents: u64,    // input
    qty: u32,                // input
) -> Option<u64>            // output or refusal
```

Contract field	Exact rule
Business rule	Totals are whole cents; quantity 1..=10,000; overflow is refused.
Rust types	u64 for money in cents, u32 for quantity, Option for refusal.
Component boundary	A pure function: no printing, no files, no global state.
Acceptance	Tests at 0, 1, 10,000, 10,001 and u64::MAX.

Common failure: Ambiguous rule. What to do: Ask the business owner (or the AI, then verify) before coding: is 0 an error or a no-op?

How you know it works: Every requirement ID appears in a test name or a trace table row.

The Rust toolchain: rustup, cargo and rustc (Slides 20-22)

rustup: Installs and updates Rust toolchains and components.

cargo: One command for new, build, run, test, fmt, clippy and doc.

rustc: The compiler; cargo calls it for you with the right flags.

stable: The course uses the stable channel; record the version as evidence.

```
# install once (macOS/Linux; Windows uses rustup-init.exe)
curl https://sh.rustup.rs -sSf | sh
rustup component add clippy rustfmt
```

```
rustc --version      # the compiler
cargo --version      # build tool + package manager
cargo new stockpilot # create a package
cargo run            # build and run
cargo test           # build and run tests
```

Contract field	Exact rule
Install	rustup from rustup.rs; Windows also needs the MSVC build tools.
Editor	Visual Studio Code with the rust-analyzer extension.
Components	clippy (lints) and rustfmt (formatting) via rustup component add.
Evidence	rustc --version output is recorded in every verification record.

Common failure: cargo: command not found. What to do: Open a new terminal so PATH includes ~/.cargo/bin, or run the rustup installer again.

How you know it works: rustc --version and cargo --version both print a version in the VS Code terminal.

Anatomy of a Cargo package (Slides 23-25)

Cargo.toml: The manifest: package metadata and dependencies.

lib.rs: Holds the reusable, testable components.

main.rs: Only parses arguments, calls the library and prints results.

tests/: Each file is compiled as a separate crate that uses the public API.

```
activity01/
  Cargo.toml      # name, edition, dependencies
  Cargo.lock      # exact resolved versions
  src/
    lib.rs        # library crate: the logic
    main.rs       # binary crate: thin entry point
  tests/
    acceptance.rs # integration tests (public API)
  target/         # build output (never commit)
```

Contract field	Exact rule
Package	One Cargo.toml; can contain one library and several binaries.
Crate	A compilation unit: lib.rs or main.rs and its modules.
Edition	edition = "2021" fixes the language rules the crate uses.
Build output	target/ is generated; add it to .gitignore.

Common failure: Logic hidden in main.rs. What to do: Move it into lib.rs so integration tests can call it directly.

How you know it works: cargo test builds lib, bin and tests; tests only use pub items.

Variables are immutable by default (Slides 26-28)

let: A binding cannot change unless you opt in with mut.

mut: Makes every place that changes state easy to find in review.

const: Named, typed constant; UPPER_SNAKE_CASE by convention.

Shadowing: Re-using a name with let creates a new variable, often of a new type.

```
let qty = 4;                // immutable binding
let mut stock = 10;         // mutable binding
stock -= qty;               // allowed: declared mut

const MAX_QTY: u32 = 10_000; // compile-time constant

let input = " 42 ";
let input: u32 = input.trim().parse().unwrap_or(0);
// shadowing: same name, new type and value
```

Contract field	Exact rule
Default	Immutable: accidental writes are compile errors (E0384).
Constants	Must have an explicit type; evaluated at compile time.
Naming	snake_case for variables and functions, CamelCase for types.
Review rule	Every mut should be necessary; clippy flags unused mut.

Common failure: error[E0384]: cannot assign twice. What to do: Add mut only if the value truly changes; otherwise create a new binding.

How you know it works: The code compiles with no unused_mut warnings under clippy.

Choose integer types deliberately (Slides 29-31)

Money: Store cents in an integer; floats cannot represent 0.10 exactly.

Widening: u64::from(qty) converts losslessly before multiplying.

checked_*: Returns None instead of overflowing.

try_from: Narrowing conversions return Result because they can fail.

```
let unit: u64 = 250;          // cents, never float
let qty: u32 = 4;             // cannot be negative
let total = unit * u64::from(qty); // widen, then multiply

u64::MAX.checked_mul(2)       // None
250u64.checked_mul(4)        // Some(1000)
3u32.checked_sub(5)           // None
(-4i64).abs()                 // 4
u32::try_from(-1i64)          // Err(..)
```

Contract field	Exact rule
u32	0..=4,294,967,295: quantities and counts.
u64	0..=18.4 quintillion: money in cents and large totals.
i64	Signed: adjustments that can be negative.

Contract field	Exact rule
Overflow policy	Debug builds panic, release builds wrap: so use <code>checked_*</code> explicitly.

Common failure: attempt to multiply with overflow (panic). What to do: Replace `*` with `checked_mul` and return `None` or an error to the caller.

How you know it works: Tests at `u64::MAX` and `u32::MAX` prove the overflow policy.

Functions: the signature is the contract (Slides 32-34)

Parameters: Every parameter has an explicit type; nothing is inferred at the boundary.

Return type: The arrow type tells callers what they must handle.

Expression body: The final expression without `;` is the value returned.

`pub`: Only `pub` functions are part of the crate API.

```
pub fn line_total(
    unit_price_cents: u64,
    qty: u32,
) -> Option<u64> {
    if qty == 0 || qty > MAX_QTY {
        return None; // early return
    }
    unit_price_cents.checked_mul(u64::from(qty))
    // last expression, no semicolon = return value
}
```

Contract field	Exact rule
Inputs	<code>unit_price_cents: u64</code> , <code>qty: u32</code> .
Output	<code>Some(total)</code> or <code>None</code> , never a panic.
Side effects	<code>None</code> : no printing, files or globals.
Tests	Normal, zero, maximum, above maximum and overflow.

Common failure: mismatched types: expected `Option<u64>`, found `u64`. What to do: Wrap the success value in `Some(...)` or return the `checked_*` result directly.

How you know it works: Five boundary tests in Activity 1 pass.

String and `&str`: owned text and borrowed text (Slides 35-37)

`String`: Owns growable UTF-8 text on the heap.

`&str`: A borrowed view of text; the preferred parameter type.

Returning `String`: Needed only when the function creates new text.

UTF-8: Byte length and character count differ for non-ASCII text.

```
let owned: String = String::from("abc-1234");
let view: &str = &owned; // borrow, no copy
let trimmed: &str = " ABC-1234 ".trim();

fn is_valid_sku(code: &str) -> bool { /* reads */ }
fn normalise_sku(code: &str) -> Option<String> {
    let upper = code.trim().to_ascii_uppercase(); // new String
    is_valid_sku(&upper).then_some(upper)
}
```

Contract field	Exact rule
Parameter	Accept &str so callers can pass literals or &String.
Return	String when new text is produced; &str when returning part of an input.
Validation	ASCII rules can use bytes safely; general text should use chars().
Tests	Include whitespace, lower case and a non-ASCII letter.

Common failure: expected &str, found String. What to do: Pass a borrow: &my_string, or change the parameter to &str.

How you know it works: Activity 2 tests pass, including "ÄBC-1234" being rejected.

Ownership moves; borrowing lends (Slides 38-40)

Owner: Every value has exactly one owner; it is dropped when the owner goes out of scope.

&T: Any number of readers at once; nobody may change the value.

&mut T: Exactly one writer and no readers at the same time.

Move: Passing by value transfers ownership; the old name cannot be used.

```
let items = vec![cable, charger];
```

```
let total = total_quantity(&items);    // shared borrow
restock(&mut items[0], 3);             // exclusive borrow
let kept = in_stock(items);            // move: items is gone
```

```
// println!("{:?}", items);
// error[E0382]: borrow of moved value: `items`
```

Contract field	Exact rule
Read only	fn f(items: &[Item]) - caller keeps ownership.
Change in place	fn f(item: &mut Item) - caller sees the change.
Consume	fn f(items: Vec<Item>) -> Vec<Item> - caller hands the value over.
Clone	An explicit copy; justify it in review because it costs memory and time.

Common failure: error[E0382]: borrow of moved value. What to do: Borrow instead of moving (&items), or clone only if two owners are truly needed.

How you know it works: Activity 3 compiles with no clone() and all five tests pass.

Activity 1: Turn a specification into a Rust function (Slides 41-45)

Folder: labs/activity-01-line-total. Maps to K1/A1. Suggested time: 25 minutes.

Goal and scenario: StockPilot must price an order line. The specification limits quantity to 1..10,000 and forbids a silently wrapped total. You create the Cargo package, write the function and prove it with tests.

Exact contract: line_total(unit_price_cents: u64, qty: u32) -> Option<u64>. qty 0 or above MAX_QTY (10,000) returns None; an overflowing product returns None; otherwise Some(total cents).

Step-by-step: Activity 1

1. Read the specification. Read the Goal and Exact contract sections below and the files in data/ (if any). Write down each input, its type and valid range, the failure behaviour and the boundary values before you open any code.
2. Open the activity in VS Code. Open this activity folder in Visual Studio Code with the rust-analyzer extension enabled. Confirm the toolchain with `rustc --version` and `cargo --version` in the integrated terminal.
3. Run the reference solution. In the terminal run: `cd solution && cargo test`. Confirm the result line shows every test passing. This proves your toolchain works and shows the target behaviour. (cargo test reports 5 passed; cargo run prints "4 x 250 cents = 1000 cents".)
4. Run the starter and read the failures. Run: `cd ../starter && cargo test`. Every test that calls an unfinished function fails with "not yet implemented" from `todo!()`. This is expected: the tests are the specification you will implement against.
5. Plan with the AI assistant. Paste the "Plan before code" prompt from PROMPTS.md into your AI coding assistant (for example GitHub Copilot Chat, Claude or ChatGPT). Compare its edge cases with your own list from step 1 and record agreements and disagreements in docs/review-record.md.
6. Vibe-code the implementation. Use the "Generate with AI" prompt to generate code for starter/src/lib.rs. Review the proposal before accepting it: public signatures unchanged, no unsafe, no unwrap() or expect() on input data, no new crates, and every line explained. Reject or edit anything you cannot justify.
7. Test until green. Run cargo test after each accepted change. If a test fails or the code does not compile, use the "Repair from evidence" prompt with the exact cargo output. Never edit expected values in tests to make them pass.
8. Apply the quality gates. Run every gate from the starter folder: `cargo test` ; `cargo run` ; `cargo clippy --all-targets -- -D warnings` ; `cargo fmt --check`. Fix each clippy warning and formatting difference; these are the organisation's coding standards.
9. Add your own boundary test. Predict one more boundary case from the specification and add it to the test file in starter/tests/. Run it against both starter and solution. Stretch: Add a discount_percent parameter (0..=100) and two boundary tests for 0 and 100.
10. Record the evidence. Complete docs/verification-record.md with the commands you ran, the exact result lines (for example "test result: ok. 5 passed"), the AI proposals you accepted or rejected and why. Compare your code with solution/ only after your own tests pass.

Commands: Activity 1

```
cd solution && cargo test
cd ../starter
cargo test
cargo run
cargo clippy --all-targets -- -D warnings
cargo fmt --check
```

Vibe-coding prompts: Activity 1

Plan before code: Act as a senior Rust reviewer. Specification: `line_total(unit_price_cents: u64, qty: u32) -> Option<u64>`. `qty 0 or above MAX_QTY (10,000)` returns `None`; an overflowing product returns `None`; otherwise `Some(total cents)`. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI: Implement "Turn a specification into a Rust function" in starter/src so that the supplied tests pass. Keep the `u64/u32` types and the `Option` return; use checked arithmetic only.

Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence: Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

The same prompts are in labs/activity-01-line-total/PROMPTS.md and Vibe-Coding-Prompts.pdf.

Reference solution: Activity 1

solution/src/lib.rs

```
//! Activity 1: price one order line from a written specification.

/// Largest quantity a single order line may request.
pub const MAX_QTY: u32 = 10_000;

/// Returns the line total in cents.
///
/// Returns `None` when `qty` is outside `1..=MAX_QTY` or when the
/// multiplication would overflow `u64`.
pub fn line_total(unit_price_cents: u64, qty: u32) -> Option<u64> {
    if qty == 0 || qty > MAX_QTY {
        return None;
    }
    unit_price_cents.checked_mul(u64::from(qty))
}
```

solution/src/main.rs

```
use activity01::line_total;

fn main() {
    let unit_price_cents: u64 = 250;
    let qty: u32 = 4;
    match line_total(unit_price_cents, qty) {
        Some(total) => {
            println!("{qty} x {unit_price_cents} cents = {total} cents")
        }
        None => println!("rejected: quantity or total out of range"),
    }
}
```

Acceptance tests: Activity 1

tests/acceptance.rs (identical in starter/ and solution/)

```
use activity01::{line_total, MAX_QTY};

#[test]
fn normal_line_total() {
    assert_eq!(line_total(250, 4), Some(1_000));
}

#[test]
fn zero_quantity_is_rejected() {
    assert_eq!(line_total(250, 0), None);
}

#[test]
fn maximum_quantity_is_accepted() {
    assert_eq!(line_total(1, MAX_QTY), Some(10_000));
}
```

```
#[test]
fn quantity_above_maximum_is_rejected() {
    assert_eq!(line_total(1, MAX_QTY + 1), None);
}

#[test]
fn overflow_is_rejected_not_wrapped() {
    assert_eq!(line_total(u64::MAX, 2), None);
}
```

Expected result: cargo test reports 5 passed; cargo run prints "4 x 250 cents = 1000 cents".

Verified before release: solution 5 tests passed; starter 0 passed / 5 failed as intended.

Stretch task: Add a discount_percent parameter (0..=100) and two boundary tests for 0 and 100.

Deliverables: your completed starter crate passing every gate, one extra boundary test, docs/review-record.md and docs/verification-record.md.

Activity 2: Validate a product code with strings (Slides 46-50)

Folder: labs/activity-02-sku-validation. Maps to K1/A1. Suggested time: 25 minutes.

Goal and scenario: Product codes arrive from scanners and spreadsheets with stray spaces and lower-case letters. StockPilot accepts only the SKU format AAA-9999 and normalises valid input to upper case.

Exact contract: is_valid_sku(&str) -> bool: after trimming, exactly 3 ASCII upper-case letters, "-", 4 ASCII digits. normalise_sku(&str) -> Option<String>: trimmed upper-case SKU when valid, else None.

Step-by-step: Activity 2

1. Read the specification. Read the Goal and Exact contract sections below and the files in data/ (if any). Write down each input, its type and valid range, the failure behaviour and the boundary values before you open any code.
2. Open the activity in VS Code. Open this activity folder in Visual Studio Code with the rust-analyzer extension enabled. Confirm the toolchain with rustc --version and cargo --version in the integrated terminal.
3. Run the reference solution. In the terminal run: cd solution && cargo test. Confirm the result line shows every test passing. This proves your toolchain works and shows the target behaviour. (cargo test reports 6 passed, including the non-ASCII and whitespace cases.)
4. Run the starter and read the failures. Run: cd ../starter && cargo test. Every test that calls an unfinished function fails with "not yet implemented" from todo!(). This is expected: the tests are the specification you will implement against.
5. Plan with the AI assistant. Paste the "Plan before code" prompt from PROMPTS.md into your AI coding assistant (for example GitHub Copilot Chat, Claude or ChatGPT). Compare its edge cases with your own list from step 1 and record agreements and disagreements in docs/review-record.md.
6. Vibe-code the implementation. Use the "Generate with AI" prompt to generate code for starter/src/lib.rs. Review the proposal before accepting it: public signatures unchanged, no unsafe, no unwrap() or expect() on input data, no new crates, and every line explained. Reject or edit anything you cannot justify.
7. Test until green. Run cargo test after each accepted change. If a test fails or the code does not compile, use the "Repair from evidence" prompt with the exact cargo output. Never edit expected values in tests to make them pass.

8. Apply the quality gates. Run every gate from the starter folder: `cargo test ; cargo clippy --all-targets -- -D warnings ; cargo fmt --check`. Fix each clippy warning and formatting difference; these are the organisation's coding standards.

9. Add your own boundary test. Predict one more boundary case from the specification and add it to the test file in `starter/tests/`. Run it against both starter and solution. Stretch: Add a `category_prefix(&str) -> Option<&str>` that returns the first three letters of a valid SKU.

10. Record the evidence. Complete `docs/verification-record.md` with the commands you ran, the exact result lines (for example "test result: ok. 5 passed"), the AI proposals you accepted or rejected and why. Compare your code with solution/ only after your own tests pass.

Commands: Activity 2

```
cd solution && cargo test
cd ../starter
cargo test
cargo clippy --all-targets -- -D warnings
cargo fmt --check
```

Vibe-coding prompts: Activity 2

Plan before code: Act as a senior Rust reviewer. Specification: `is_valid_sku(&str) -> bool`: after trimming, exactly 3 ASCII upper-case letters, "-", 4 ASCII digits. `normalise_sku(&str) -> Option<String>`: trimmed upper-case SKU when valid, else None. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI: Implement "Validate a product code with strings" in `starter/src` so that the supplied tests pass. Borrow `&str` inputs, return an owned `String` only from `normalise_sku`, and do not use regex crates. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence: Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

The same prompts are in `labs/activity-02-sku-validation/PROMPTS.md` and `Vibe-Coding-Prompts.pdf`.

Reference solution: Activity 2

`solution/src/lib.rs`

```
//! Activity 2: product-code validation with &str and String.

/// Returns true when `code`, after trimming, has the form `AAA-9999`.
pub fn is_valid_sku(code: &str) -> bool {
    let bytes = code.trim().as_bytes();
    bytes.len() == 8
        && bytes[..3].iter().all(u8::is_ascii_uppercase)
        && bytes[3] == b'-'
        && bytes[4..].iter().all(u8::is_ascii_digit)
}

/// Returns the trimmed, upper-case SKU when it is valid.
pub fn normalise_sku(code: &str) -> Option<String> {
    let upper = code.trim().to_ascii_uppercase();
    if is_valid_sku(&upper) {
        Some(upper)
    } else {
        None
    }
}
```

```
}
}
```

Acceptance tests: Activity 2

tests/acceptance.rs (identical in starter/ and solution/)

```
use activity02::{is_valid_sku, normalise_sku};

#[test]
fn accepts_the_documented_format() {
    assert!(is_valid_sku("ABC-1234"));
}

#[test]
fn trims_surrounding_whitespace() {
    assert!(is_valid_sku("  ABC-1234\n"));
}

#[test]
fn rejects_lower_case_letters() {
    assert!(is_valid_sku("abc-1234"));
}

#[test]
fn rejects_wrong_layout_and_letters_in_digits() {
    assert!(is_valid_sku("AB-12345"));
    assert!(is_valid_sku("ABC-12A4"));
    assert!(is_valid_sku(""));
}

#[test]
fn rejects_non_ascii_letters() {
    assert!(is_valid_sku("ÄBC-1234"));
}

#[test]
fn normalises_valid_input_to_upper_case() {
    assert_eq!(normalise_sku(" abc-1234 "), Some("ABC-1234".to_string()));
    assert_eq!(normalise_sku("bad"), None);
}
```

Expected result: cargo test reports 6 passed, including the non-ASCII and whitespace cases.

Verified before release: solution 6 tests passed; starter 0 passed / 6 failed as intended.

Stretch task: Add a category_prefix(&str) -> Option<&str> that returns the first three letters of a valid SKU.

Deliverables: your completed starter crate passing every gate, one extra boundary test, docs/review-record.md and docs/verification-record.md.

Activity 3: Share inventory records with ownership and borrowing (Slides 51-55)

Folder: labs/activity-03-ownership-borrowing. Maps to K1/A1 K3/A3. Suggested time: 25 minutes.

Goal and scenario: Three StockPilot functions handle the same item list differently: one only reads it, one changes one item and one consumes the list. You choose the parameter types that express those intentions.

Exact contract: Item { sku, name, qty: u32 }. total_quantity(&[Item]) -> u64 reads; restock(&mut Item, u32) -> Option<u32> changes one item and leaves it unchanged on overflow; in_stock(Vec<Item>) -> Vec<Item> consumes the list.

Step-by-step: Activity 3

1. Read the specification. Read the Goal and Exact contract sections below and the files in data/ (if any). Write down each input, its type and valid range, the failure behaviour and the boundary values before you open any code.
2. Open the activity in VS Code. Open this activity folder in Visual Studio Code with the rust-analyzer extension enabled. Confirm the toolchain with `rustc --version` and `cargo --version` in the integrated terminal.
3. Run the reference solution. In the terminal run: `cd solution && cargo test`. Confirm the result line shows every test passing. This proves your toolchain works and shows the target behaviour. (cargo test reports 5 passed; the overflow test proves the item is unchanged.)
4. Run the starter and read the failures. Run: `cd ../starter && cargo test`. Every test that calls an unfinished function fails with "not yet implemented" from `todo!()`. This is expected: the tests are the specification you will implement against.
5. Plan with the AI assistant. Paste the "Plan before code" prompt from PROMPTS.md into your AI coding assistant (for example GitHub Copilot Chat, Claude or ChatGPT). Compare its edge cases with your own list from step 1 and record agreements and disagreements in docs/review-record.md.
6. Vibe-code the implementation. Use the "Generate with AI" prompt to generate code for starter/src/lib.rs. Review the proposal before accepting it: public signatures unchanged, no unsafe, no `unwrap()` or `expect()` on input data, no new crates, and every line explained. Reject or edit anything you cannot justify.
7. Test until green. Run cargo test after each accepted change. If a test fails or the code does not compile, use the "Repair from evidence" prompt with the exact cargo output. Never edit expected values in tests to make them pass.
8. Apply the quality gates. Run every gate from the starter folder: `cargo test ; cargo clippy --all-targets -- -D warnings ; cargo fmt --check`. Fix each clippy warning and formatting difference; these are the organisation's coding standards.
9. Add your own boundary test. Predict one more boundary case from the specification and add it to the test file in starter/tests/. Run it against both starter and solution. Stretch: Add `rename(item: &mut Item, new_name: &str)` and a test proving the SKU is unchanged.
10. Record the evidence. Complete docs/verification-record.md with the commands you ran, the exact result lines (for example "test result: ok. 5 passed"), the AI proposals you accepted or rejected and why. Compare your code with solution/ only after your own tests pass.

Commands: Activity 3

```
cd solution && cargo test
cd ../starter
cargo test
cargo clippy --all-targets -- -D warnings
cargo fmt --check
```

Vibe-coding prompts: Activity 3

Plan before code: Act as a senior Rust reviewer. Specification: `Item { sku, name, qty: u32 }`. `total_quantity(&[Item]) -> u64` reads; `restock(&mut Item, u32) -> Option<u32>` changes one item and leaves it unchanged on overflow; `in_stock(Vec<Item>) -> Vec<Item>` consumes the list. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI: Implement "Share inventory records with ownership and borrowing" in starter/src so that the supplied tests pass. Choose `&`, `&mut` or by-value parameters to match each function's intent; avoid `clone()` unless you justify it. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence: Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

The same prompts are in labs/activity-03-ownership-borrowing/PROMPTS.md and Vibe-Coding-Prompts.pdf.

Reference solution: Activity 3

solution/src/lib.rs

```
//! Activity 3: ownership, borrowing and mutable borrowing.

/// One stock record.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Item {
    /// Product code.
    pub sku: String,
    /// Display name.
    pub name: String,
    /// Units on hand.
    pub qty: u32,
}

impl Item {
    /// Creates an owned item from borrowed text.
    pub fn new(sku: &str, name: &str, qty: u32) -> Self {
        Item {
            sku: sku.to_string(),
            name: name.to_string(),
            qty,
        }
    }
}

/// Borrows the items immutably; the caller keeps ownership.
pub fn total_quantity(items: &[Item]) -> u64 {
    items.iter().map(|item| u64::from(item.qty)).sum()
}

/// Borrows one item mutably and returns its new quantity.
///
/// On overflow the item is left unchanged and `None` is returned.
pub fn restock(item: &mut Item, amount: u32) -> Option<u32> {
    let new_qty = item.qty.checked_add(amount)?;
    item.qty = new_qty;
    Some(new_qty)
}

/// Takes ownership of the list and returns only the items in stock.
pub fn in_stock(items: Vec<Item>) -> Vec<Item> {
    items.into_iter().filter(|item| item.qty > 0).collect()
}
```

Acceptance tests: Activity 3

tests/acceptance.rs (identical in starter/ and solution/)

```
use activity03::{in_stock, restock, total_quantity, Item};
```

```

fn sample() -> Vec<Item> {
    vec![
        Item::new("ABC-0001", "USB-C cable", 5),
        Item::new("ABC-0002", "Wall charger", 0),
        Item::new("ABC-0003", "Phone case", 7),
    ]
}

#[test]
fn total_borrows_without_taking_ownership() {
    let items = sample();
    assert_eq!(total_quantity(&items), 12);
    assert_eq!(items.len(), 3); // still usable: it was only borrowed
}

#[test]
fn empty_total_is_zero() {
    assert_eq!(total_quantity(&[]), 0);
}

#[test]
fn restock_changes_the_item_through_a_mutable_borrow() {
    let mut item = Item::new("ABC-0001", "USB-C cable", 5);
    assert_eq!(restock(&mut item, 3), Some(8));
    assert_eq!(item.qty, 8);
}

#[test]
fn restock_overflow_leaves_the_item_unchanged() {
    let mut item = Item::new("ABC-0001", "USB-C cable", u32::MAX);
    assert_eq!(restock(&mut item, 1), None);
    assert_eq!(item.qty, u32::MAX);
}

#[test]
fn in_stock_consumes_the_list_and_filters_it() {
    let kept = in_stock(sample());
    let skus: Vec<&str> = kept.iter().map(|item| item.sku.as_str()).collect();
    assert_eq!(skus, ["ABC-0001", "ABC-0003"]);
}

```

Expected result: cargo test reports 5 passed; the overflow test proves the item is unchanged.

Verified before release: solution 5 tests passed; starter 0 passed / 5 failed as intended.

Stretch task: Add `rename(item: &mut Item, new_name: &str)` and a test proving the SKU is unchanged.

Deliverables: your completed starter crate passing every gate, one extra boundary test, docs/review-record.md and docs/verification-record.md.

Topic 2: AI Vibe Coding for Rust Application Development (Slides 56-95)

K2/A2 / LO2: Apply programming methodologies and tools for software creation (K2/A2).

The vibe-coding loop for Rust (Slides 57-59)

Intent: Describe the outcome in plain language, as a user would.

Contract: Pin down types, ranges and errors so the output can be judged.

Compiler + tests: Rust rejects many wrong programs before they run.

Human review: You own the code: accept only what you can explain.

1. intent "parse add/remove/list commands"
2. contract types, ranges, errors, tests
3. prompt bounded request to the AI
4. generate AI proposes code
5. compile cargo build (the first reviewer)
6. test cargo test (the specification)
7. review read, question, accept or reject
8. record decision + evidence

Contract field	Exact rule
Speed	AI drafts in seconds; the loop turns drafts into trusted code.
Iteration	Small prompts and small diffs are easier to review.
Stop rule	Never accept code that fails a gate or that you cannot explain.
Record	Keep prompt, proposal, decision and test result for each change.

Common failure: AI output looks right but was never run. What to do: Run cargo test and clippy before accepting; paste failures back as evidence.

How you know it works: Each activity review record shows prompt, decision and the passing test result.

Write a bounded prompt (Slides 60-62)

Role: Sets the expected quality bar and tone.

Contract: The exact signature and rules to satisfy.

Limits: What the AI must not do: crates, unsafe, signature changes.

Output: Ask for tests and reasons, not only code.

Role: Act as a senior Rust reviewer.
Context: StockPilot command parser, Rust 2021, std only.
Contract: parse_command(&[&str]) -> Result<Command, String>
list | add <sku> <qty> | remove <sku> <qty>
qty: u32 and >= 1
Limits: no new crates, no unsafe, no unwrap on input,
keep the public signature.
Output: code + 4 boundary tests + one-line reasons.

Contract field	Exact rule
Include	Specification, relevant file, exact cargo output.
Exclude	Secrets, API keys, personal data, assessment answers.
Size	One function or one fix per prompt.
Follow-up	Ask "which edge case did you not test?" before accepting.

Common failure: AI invents a crate or API. What to do: Reject; restate the limits and ask for a std-only version, then compile to confirm.

How you know it works: The accepted proposal compiles, passes the tests and respects every limit.

The compiler is your first reviewer (Slides 63-65)

Error code: E0382 can be looked up with `rustc --explain E0382`.

Spans: The arrows show where the value moved and where it was used.

Help lines: Often suggest the exact fix (borrow, clone, add mut).

AI use: Paste the full message; ask for the cause before the fix.

```
error[E0382]: borrow of moved value: `items`
--> src/main.rs:9:22
   |
5 |     let items = vec![cable, charger];
   |         ----- move occurs because `items` has type
   |             `Vec<Item>`, which does not implement Copy
7 |     let kept = in_stock(items);
   |                                   ----- value moved here
9 |     println!("{:?}", items);
   |                   ^^^^^ value borrowed here after move
```

Contract field	Exact rule
Read order	First error first; later errors are often consequences.
Explain	<code>rustc --explain E0XXX</code> gives a worked example.
Fix scope	Smallest change that keeps the contract.
Evidence	Record the error and the fix in the review record.

Common failure: Many errors after one AI change. What to do: Revert to the last green state, apply the change in smaller steps and rebuild.

How you know it works: cargo build reports zero errors and zero warnings.

Review AI-generated Rust before accepting it (Slides 66-68)

unwrap/expect: Acceptable in tests; in library code they turn bad input into a crash.

unsafe: Removes compiler guarantees; not needed for this course.

clone: Often hides an ownership question; ask why it is needed.

New crates: Add supply-chain risk; only with explicit approval.

```
// Red flags in a proposal
let qty: u32 = args[2].parse().unwrap(); // panics on bad input
unsafe { ... }                          // never needed here
let copy = items.clone();                // unjustified clone
use clap::Parser;                        // new crate, not allowed
```

```
// Accept when
cargo test # all pass
cargo clippy --all-targets -- -D warnings # clean
```

Contract field	Exact rule
Signatures	Public API unchanged unless the specification changed.
Errors	Invalid input returns Err/None, never panics.
Tests	Every boundary in the specification is tested.
Explainability	You can explain every line to a colleague.

Common failure: Proposal passes tests but uses unwrap on input. What to do: Reject; ask for `map_err/?` so bad input returns an error the caller can handle.

How you know it works: The review record lists at least one rejected or edited suggestion with a reason.

Modules and visibility organise a crate (Slides 69-71)

mod: Declares a module; the file name matches the module name.

Privacy: Everything is private unless marked `pub`.

Paths: `crate::` starts from the crate root; `super::` goes up one level.

use: Brings a path into scope so it can be called by its short name.

```
// src/lib.rs
pub mod money;           // loads src/money.rs
pub mod report;          // loads src/report.rs

// src/report.rs
use crate::money::format_cents;

pub fn report_line(name: &str, qty: u32, unit: u64)
    -> Option<String> { /* ... */ }
fn helper() {}           // private to this module
```

Contract field	Exact rule
pub	Visible to other crates, including your tests/ folder.
pub(crate)	Visible inside this crate only.
Module per concern	money formats currency; report builds lines.
API check	Integration tests compile only against pub items.

Common failure: `error[E0603]: function is private`. What to do: Make it `pub` if it is part of the API, or test it inside the module with `#[cfg(test)]`.

How you know it works: Activity 5 tests `import activity05::money` and `activity05::report` successfully.

Cargo dependencies and Cargo.lock (Slides 72-74)

Semver: "1" means any compatible 1.x version.

Features: Opt-in capabilities such as `serde`'s `derive` macros.

Cargo.lock: Exact versions used; commit it for applications and courses.

crates.io: The public registry; check downloads, docs and maintenance.

```
# Cargo.toml
[dependencies]
serde = { version = "1", features = ["derive"] }
serde_json = "1"

cargo add serde --features derive # edits Cargo.toml
cargo build                       # downloads + locks
cargo tree                       # show dependency graph
cargo update                     # move within semver range
```

Contract field	Exact rule
Approval	Every new crate is a design decision recorded in review.
Reproducibility	Cargo.lock gives every learner the same versions.

Contract field	Exact rule
Offline	cargo fetch downloads everything before class.
Audit	cargo tree shows what a crate pulls in.

Common failure: failed to select a version / download error. What to do: Check the version string and network, then cargo fetch; never copy crates by hand.

How you know it works: Activity 6 builds with serde 1.x recorded in Cargo.lock and 6 tests pass.

fmt and clippy encode team practice (Slides 75-77)

rustfmt: One formatting style for everyone; no style debates in review.

clippy: Hundreds of lints for bugs, performance and idiom.

-D warnings: Treat warnings as errors so they cannot accumulate.

rustfmt.toml: Project settings, for example max_width = 80.

```
cargo fmt                                # rewrite to house style
cargo fmt --check                        # fail if not formatted
cargo clippy --all-targets -- -D warnings
```

```
warning: this `if` has identical blocks
warning: redundant clone
warning: missing documentation for a function
    = help: for further information visit
           https://rust-lang.github.io/rust-clippy/...
```

Contract field	Exact rule
Before commit	fmt --check, clippy -D warnings and test must all pass.
AI output	Run the same gates on generated code; no exceptions.
Allow a lint	Only with #[allow(...)] plus a reason in a comment.
CI	The same three commands can run in a pipeline.

Common failure: clippy fails after an AI change. What to do: Read the lint link, apply the suggested idiom, rerun tests to confirm behaviour.

How you know it works: Every activity solution passes fmt --check and clippy -D warnings.

Set up VS Code for Rust vibe coding (Slides 78-80)

rust-analyzer: Shows the same errors as the compiler while you type.

Inlay hints: Display inferred types, which helps review AI code.

Run/Test lenses: Click "Run Test" above a #[test] function.

AI panel: Keep prompts beside the code; paste exact errors.

```
Extensions
rust-analyzer          types, errors, go-to-definition
(optional) CodeLLDB    step-through debugging
your AI assistant      Copilot / Claude / ChatGPT
```

```
Workspace habits
open the crate folder  (the one with Cargo.toml)
inlay hints on         see inferred types
terminal: cargo test   after every accepted change
```

Contract field	Exact rule
Folder	Open starter/ or solution/ so rust-analyzer finds Cargo.toml.
Save	Format on save can call rustfmt automatically.
Terminal	Use the integrated terminal for cargo commands.
Privacy	Do not paste secrets or assessment answers into AI tools.

Common failure: rust-analyzer: failed to find a workspace. What to do: Open the folder that contains Cargo.toml, then reload the window.

How you know it works: Hovering a variable shows its type and cargo test runs from the terminal.

Activity 4: Vibe-code a command parser (Slides 81-85)

Folder: labs/activity-04-cli-commands. Maps to K2/A2. Suggested time: 25 minutes.

Goal and scenario: Store staff will type commands such as "add ABC-0001 5". You describe the command language to an AI assistant, review the generated parser, and keep only code that passes the acceptance tests.

Exact contract: `parse_command(&[&str]) -> Result<Command, String>`. Accepts exactly: `list | add <sku> <qty> | remove <sku> <qty>`. `qty` must parse as `u32` and be at least 1. Anything else returns `Err` with a usage or reason message.

Step-by-step: Activity 4

1. Read the specification. Read the Goal and Exact contract sections below and the files in `data/` (if any). Write down each input, its type and valid range, the failure behaviour and the boundary values before you open any code.
2. Open the activity in VS Code. Open this activity folder in Visual Studio Code with the rust-analyzer extension enabled. Confirm the toolchain with `rustc --version` and `cargo --version` in the integrated terminal.
3. Run the reference solution. In the terminal run: `cd solution && cargo test`. Confirm the result line shows every test passing. This proves your toolchain works and shows the target behaviour. (cargo test reports 5 passed; cargo run -- add ABC-0001 5 prints `Add { sku: "ABC-0001", qty: 5 }`.)
4. Run the starter and read the failures. Run: `cd ../starter && cargo test`. Every test that calls an unfinished function fails with "not yet implemented" from `todo!()`. This is expected: the tests are the specification you will implement against.
5. Plan with the AI assistant. Paste the "Plan before code" prompt from `PROMPTS.md` into your AI coding assistant (for example GitHub Copilot Chat, Claude or ChatGPT). Compare its edge cases with your own list from step 1 and record agreements and disagreements in `docs/review-record.md`.
6. Vibe-code the implementation. Use the "Generate with AI" prompt to generate code for `starter/src/lib.rs`. Review the proposal before accepting it: public signatures unchanged, no `unsafe`, no `unwrap()` or `expect()` on input data, no new crates, and every line explained. Reject or edit anything you cannot justify.
7. Test until green. Run cargo test after each accepted change. If a test fails or the code does not compile, use the "Repair from evidence" prompt with the exact cargo output. Never edit expected values in tests to make them pass.

8. Apply the quality gates. Run every gate from the starter folder: `cargo test` ; `cargo run -- add ABC-0001 5` ; `cargo clippy --all-targets -- -D warnings` ; `cargo fmt --check`. Fix each clippy warning and formatting difference; these are the organisation's coding standards.
9. Add your own boundary test. Predict one more boundary case from the specification and add it to the test file in `starter/tests/`. Run it against both starter and solution. Stretch: Ask the AI to add "count <sku>"; accept it only with a new test for the missing-SKU case.
10. Record the evidence. Complete `docs/verification-record.md` with the commands you ran, the exact result lines (for example "test result: ok. 5 passed"), the AI proposals you accepted or rejected and why. Compare your code with solution/ only after your own tests pass.

Commands: Activity 4

```
cd solution && cargo test
cd ../starter
cargo test
cargo run -- add ABC-0001 5
cargo clippy --all-targets -- -D warnings
cargo fmt --check
```

Vibe-coding prompts: Activity 4

Plan before code: Act as a senior Rust reviewer. Specification: `parse_command(&[&str]) -> Result<Command, String>`. Accepts exactly: `list | add <sku> <qty> | remove <sku> <qty>`. `qty` must parse as `u32` and be at least 1. Anything else returns `Err` with a usage or reason message. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI: Implement "Vibe-code a command parser" in `starter/src` so that the supplied tests pass. Generate the parser from the command grammar only; reject any suggestion that adds crates such as `clap`. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence: Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

The same prompts are in `labs/activity-04-cli-commands/PROMPTS.md` and `Vibe-Coding-Prompts.pdf`.

Reference solution: Activity 4

`solution/src/lib.rs`

```
//! Activity 4: a command parser produced by vibe coding and human review.

/// A validated StockPilot command.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum Command {
    /// Receive stock.
    Add {
        /// Product code.
        sku: String,
        /// Units received.
        qty: u32,
    },
    /// Remove stock.
    Remove {
        /// Product code.
        sku: String,
        /// Units removed.
        qty: u32,
    },
}
```

```

    },
    /// List all stock.
    List,
}

/// Parses command-line words into a [`Command`].
pub fn parse_command(args: &[&str]) -> Result<Command, String> {
    match args {
        ["list"] => Ok(Command::List),
        [verb @ ("add" | "remove"), sku, qty] => {
            let qty: u32 = qty
                .parse()
                .map_err(|_| format!("invalid quantity: {qty}"))?;
            if qty == 0 {
                return Err("quantity must be at least 1".to_string());
            }
            let sku = sku.to_string();
            if *verb == "add" {
                Ok(Command::Add { sku, qty })
            } else {
                Ok(Command::Remove { sku, qty })
            }
        }
        _ => {
            Err("usage: add <sku> <qty> | remove <sku> <qty> | list"
                .to_string())
        }
    }
}

```

solution/src/main.rs

```

use activity04::parse_command;
use std::process::ExitCode;

fn main() -> ExitCode {
    let args: Vec<String> = std::env::args().skip(1).collect();
    let words: Vec<&str> = args.iter().map(String::as_str).collect();
    match parse_command(&words) {
        Ok(command) => {
            println!("{command:?}");
            ExitCode::SUCCESS
        }
        Err(message) => {
            eprintln!("error: {message}");
            ExitCode::from(2)
        }
    }
}

```

Acceptance tests: Activity 4

tests/acceptance.rs (identical in starter/ and solution/)

```

use activity04::{parse_command, Command};

#[test]
fn parses_list() {
    assert_eq!(parse_command(&["list"]), Ok(Command::List));
}

#[test]
fn parses_add_and_remove() {
    assert_eq!(
        parse_command(&["add", "ABC-0001", "5"]),
        Ok(Command::Add {

```

```

        sku: "ABC-0001".into(),
        qty: 5
    })
};
assert_eq!(
    parse_command(&["remove", "ABC-0001", "2"]),
    Ok(Command::Remove {
        sku: "ABC-0001".into(),
        qty: 2
    })
);
}

#[test]
fn rejects_zero_quantity() {
    assert_eq!(
        parse_command(&["add", "ABC-0001", "0"]),
        Err("quantity must be at least 1".to_string())
    );
}

#[test]
fn rejects_non_numeric_and_negative_quantities() {
    assert!(parse_command(&["add", "ABC-0001", "five"]).is_err());
    assert!(parse_command(&["remove", "ABC-0001", "-3"]).is_err());
}

#[test]
fn rejects_unknown_verbs_missing_and_extra_words() {
    assert!(parse_command(&["delete", "ABC-0001", "1"]).is_err());
    assert!(parse_command(&["add", "ABC-0001"]).is_err());
    assert!(parse_command(&["list", "now"]).is_err());
    assert!(parse_command(&[]).is_err());
}

```

Expected result: cargo test reports 5 passed; cargo run -- add ABC-0001 5 prints Add { sku: "ABC-0001", qty: 5 }.

Verified before release: solution 5 tests passed; starter 0 passed / 5 failed as intended.

Stretch task: Ask the AI to add "count <sku>"; accept it only with a new test for the missing-SKU case.

Deliverables: your completed starter crate passing every gate, one extra boundary test, docs/review-record.md and docs/verification-record.md.

Activity 5: Organise a crate into modules (Slides 86-90)

Folder: labs/activity-05-modules-visibility. Maps to K2/A2. Suggested time: 25 minutes.

Goal and scenario: The reporting code has grown. You split StockPilot formatting into a money module and a report module, expose only what callers need, and keep the behaviour identical under test.

Exact contract: money::format_cents(u64) -> String renders "\$D.CC". report::report_line(name, qty, unit_cents) -> Option<String> renders "<name> x<qty> @ <unit> = <total>"; report::report joins lines with "\n" and fails wholly on overflow.

Step-by-step: Activity 5

1. Read the specification. Read the Goal and Exact contract sections below and the files in data/ (if any). Write down each input, its type and valid range, the failure behaviour and the boundary values before you open any code.

2. Open the activity in VS Code. Open this activity folder in Visual Studio Code with the rust-analyzer extension enabled. Confirm the toolchain with `rustc --version` and `cargo --version` in the integrated terminal.
3. Run the reference solution. In the terminal run: `cd solution && cargo test`. Confirm the result line shows every test passing. This proves your toolchain works and shows the target behaviour. (cargo test reports 4 passed; tests import `activity05::money` and `activity05::report`.)
4. Run the starter and read the failures. Run: `cd ../starter && cargo test`. Every test that calls an unfinished function fails with "not yet implemented" from `todo!()`. This is expected: the tests are the specification you will implement against.
5. Plan with the AI assistant. Paste the "Plan before code" prompt from `PROMPTS.md` into your AI coding assistant (for example GitHub Copilot Chat, Claude or ChatGPT). Compare its edge cases with your own list from step 1 and record agreements and disagreements in `docs/review-record.md`.
6. Vibe-code the implementation. Use the "Generate with AI" prompt to generate code for `starter/src/lib.rs`, `starter/src/money.rs`, `starter/src/report.rs`. Review the proposal before accepting it: public signatures unchanged, no `unsafe`, no `unwrap()` or `expect()` on input data, no new crates, and every line explained. Reject or edit anything you cannot justify.
7. Test until green. Run cargo test after each accepted change. If a test fails or the code does not compile, use the "Repair from evidence" prompt with the exact cargo output. Never edit expected values in tests to make them pass.
8. Apply the quality gates. Run every gate from the starter folder: `cargo test ; cargo clippy --all-targets -- -D warnings ; cargo fmt --check`. Fix each clippy warning and formatting difference; these are the organisation's coding standards.
9. Add your own boundary test. Predict one more boundary case from the specification and add it to the test file in `starter/tests/`. Run it against both starter and solution. Stretch: Move `format_cents` behind `pub(crate)` and explain which test import then fails to compile.
10. Record the evidence. Complete `docs/verification-record.md` with the commands you ran, the exact result lines (for example "test result: ok. 5 passed"), the AI proposals you accepted or rejected and why. Compare your code with solution/ only after your own tests pass.

Commands: Activity 5

```
cd solution && cargo test
cd ../starter
cargo test
cargo clippy --all-targets -- -D warnings
cargo fmt --check
```

Vibe-coding prompts: Activity 5

Plan before code: Act as a senior Rust reviewer. Specification: `money::format_cents(u64) -> String` renders "\$D.CC". `report::report_line(name, qty, unit_cents) -> Option<String>` renders "<name> x<qty> @ <unit> = <total>"; `report::report` joins lines with "\n" and fails wholly on overflow. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI: Implement "Organise a crate into modules" in `starter/src` so that the supplied tests pass. Keep the two-module layout and public paths; do not merge modules to make tests pass. Keep public signatures unchanged, use no `unsafe` code, and explain each design decision in one sentence.

Repair from evidence: Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

The same prompts are in labs/activity-05-modules-visibility/PROMPTS.md and Vibe-Coding-Prompts.pdf.

Reference solution: Activity 5

solution/src/lib.rs

```
//! Activity 5: a crate split into modules with explicit visibility.
```

```
pub mod money;  
pub mod report;
```

solution/src/money.rs

```
//! Currency formatting, kept separate from reporting.
```

```
/// Formats whole cents as dollars, for example `1205` becomes `"$12.05"`.  
pub fn format_cents(cents: u64) -> String {  
    format!("${}.{:02}", cents / 100, cents % 100)  
}
```

solution/src/report.rs

```
//! Report lines built on top of the money module.
```

```
use crate::money::format_cents;
```

```
/// Builds ` "<name> x<qty> @ <unit> = <total>"`, or `None` on overflow.  
pub fn report_line(name: &str, qty: u32, unit_cents: u64) -> Option<String> {  
    let total = unit_cents.checked_mul(u64::from(qty))?;  
    Some(format!(  
        "{name} x{qty} @ {} = {}",  
        format_cents(unit_cents),  
        format_cents(total)  
    ))  
}
```

```
/// Joins one report line per row; any overflow fails the whole report.  
pub fn report(rows: &[(&str, u32, u64)]) -> Option<String> {  
    let lines: Option<Vec<String>> = rows  
        .iter()  
        .map(|&(name, qty, unit)| report_line(name, qty, unit))  
        .collect();  
    lines.map(|lines| lines.join("\n"))  
}
```

Acceptance tests: Activity 5

tests/acceptance.rs (identical in starter/ and solution/)

```
use activity05::money::format_cents;  
use activity05::report::{report, report_line};  
  
#[test]  
fn formats_cents_with_two_decimal_places() {  
    assert_eq!(format_cents(0), "$0.00");  
    assert_eq!(format_cents(5), "$0.05");  
    assert_eq!(format_cents(1205), "$12.05");  
    assert_eq!(format_cents(100_000), "$1000.00");  
}  
  
#[test]  
fn builds_one_report_line() {  
    assert_eq!(  
        report_line("Cable", 3, 250),  
        Some("Cable x3 @ $2.50 = $7.50".to_string())  
    );  
}
```

```

}

#[test]
fn joins_lines_in_order() {
    let rows = [("Cable", 3, 250), ("Charger", 1, 2499)];
    assert_eq!(
        report(&rows),
        Some(
            "Cable x3 @ $2.50 = $7.50\nCharger x1 @ $24.99 = $24.99"
                .to_string()
        )
    );
}

#[test]
fn overflow_fails_the_whole_report() {
    assert_eq!(report_line("Bulk", 2, u64::MAX), None);
    assert_eq!(report(&[("Cable", 1, 1), ("Bulk", 2, u64::MAX)]), None);
}

```

Expected result: cargo test reports 4 passed; tests import activity05::money and activity05::report.

Verified before release: solution 4 tests passed; starter 0 passed / 4 failed as intended.

Stretch task: Move `format_cents` behind `pub(crate)` and explain which test import then fails to compile.

Deliverables: your completed starter crate passing every gate, one extra boundary test, docs/review-record.md and docs/verification-record.md.

Activity 6: Load inventory JSON with Cargo dependencies (Slides 91-95)

Folder: labs/activity-06-cargo-serde-json. Maps to K2/A2 K4/A4. Suggested time: 25 minutes.

Goal and scenario: The purchasing team exports inventory as JSON. You add `serde` and `serde_json` with Cargo, derive the data contract, and let the type system reject malformed records at the boundary.

Exact contract: `Item { sku, name, qty: u32, unit_cents: u64 }` derives `Serialize` and `Deserialize`.
`load_items(&str) -> Result<Vec<Item>, serde_json::Error>; to_json(&[Item]) -> Result<String, _>;`
`stock_value(&[Item]) -> Option<u64>.`

Step-by-step: Activity 6

1. Read the specification. Read the Goal and Exact contract sections below and the files in `data/` (if any). Write down each input, its type and valid range, the failure behaviour and the boundary values before you open any code.
2. Open the activity in VS Code. Open this activity folder in Visual Studio Code with the `rust-analyzer` extension enabled. Confirm the toolchain with `rustc --version` and `cargo --version` in the integrated terminal.
3. Run the reference solution. In the terminal run: `cd solution && cargo test`. Confirm the result line shows every test passing. This proves your toolchain works and shows the target behaviour. (cargo test reports 6 passed; stock value of `data/inventory.json` is 65,948 cents.)
4. Run the starter and read the failures. Run: `cd ../starter && cargo test`. Every test that calls an unfinished function fails with "not yet implemented" from `todo!()`. This is expected: the tests are the specification you will implement against.
5. Plan with the AI assistant. Paste the "Plan before code" prompt from `PROMPTS.md` into your AI coding assistant (for example GitHub Copilot Chat, Claude or ChatGPT). Compare its edge cases with your own list from step 1 and record agreements and disagreements in `docs/review-record.md`.

6. Vibe-code the implementation. Use the "Generate with AI" prompt to generate code for starter/src/lib.rs. Review the proposal before accepting it: public signatures unchanged, no unsafe, no unwrap() or expect() on input data, no new crates, and every line explained. Reject or edit anything you cannot justify.

7. Test until green. Run cargo test after each accepted change. If a test fails or the code does not compile, use the "Repair from evidence" prompt with the exact cargo output. Never edit expected values in tests to make them pass.

8. Apply the quality gates. Run every gate from the starter folder: cargo test ; cargo clippy --all-targets -- -D warnings ; cargo fmt --check. Fix each clippy warning and formatting difference; these are the organisation's coding standards.

9. Add your own boundary test. Predict one more boundary case from the specification and add it to the test file in starter/tests/. Run it against both starter and solution. Stretch: Add #[serde(deny_unknown_fields)] and a test proving an extra "colour" field is rejected.

10. Record the evidence. Complete docs/verification-record.md with the commands you ran, the exact result lines (for example "test result: ok. 5 passed"), the AI proposals you accepted or rejected and why. Compare your code with solution/ only after your own tests pass.

Commands: Activity 6

```
cd solution && cargo test
cd ../starter
cargo test
cargo clippy --all-targets -- -D warnings
cargo fmt --check
```

Vibe-coding prompts: Activity 6

Plan before code: Act as a senior Rust reviewer. Specification: Item { sku, name, qty: u32, unit_cents: u64 } derives Serialize and Deserialize. load_items(&str) -> Result<Vec<Item>, serde_json::Error>; to_json(&[Item]) -> Result<String, _>; stock_value(&[Item]) -> Option<u64>. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI: Implement "Load inventory JSON with Cargo dependencies" in starter/src so that the supplied tests pass. Use serde derive only; do not hand-write JSON parsing or add other crates. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence: Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

The same prompts are in labs/activity-06-cargo-serde-json/PROMPTS.md and Vibe-Coding-Prompts.pdf.

Reference solution: Activity 6

solution/src/lib.rs

```
//! Activity 6: a JSON data contract using the serde and serde_json crates.

use serde::{Deserialize, Serialize};

/// One inventory record as exported by purchasing.
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserialize)]
pub struct Item {
    /// Product code.
```

```

    pub sku: String,
    /// Display name.
    pub name: String,
    /// Units on hand.
    pub qty: u32,
    /// Unit cost in cents.
    pub unit_cents: u64,
}

/// Parses a JSON array of items; missing or mistyped fields are errors.
pub fn load_items(json: &str) -> Result<Vec<Item>, serde_json::Error> {
    serde_json::from_str(json)
}

/// Serialises items as pretty-printed JSON.
pub fn to_json(items: &[Item]) -> Result<String, serde_json::Error> {
    serde_json::to_string_pretty(items)
}

/// Total stock value in cents, or `None` on overflow.
pub fn stock_value(items: &[Item]) -> Option<u64> {
    items.iter().try_fold(0u64, |total, item| {
        total.checked_add(item.unit_cents.checked_mul(u64::from(item.qty)))
    })
}

```

Acceptance tests: Activity 6

tests/acceptance.rs (identical in starter/ and solution/)

```

use activity06::{load_items, stock_value, to_json, Item};

const SAMPLE: &str = include_str!("../../data/inventory.json");

#[test]
fn loads_the_sample_file() {
    let items = load_items(SAMPLE).expect("sample is valid");
    assert_eq!(items.len(), 3);
    assert_eq!(items[1].name, "Wall charger");
}

#[test]
fn calculates_stock_value() {
    let items = load_items(SAMPLE).unwrap();
    assert_eq!(stock_value(&items), Some(65_948));
}

#[test]
fn missing_field_is_rejected() {
    assert!(load_items(r#"{"sku":"ABC-0009","name":"Hub","qty":1}"#).is_err());
}

#[test]
fn negative_quantity_is_rejected_by_the_type() {
    let json = r#"{"sku":"ABC-0009","name":"Hub","qty":-1,"unit_cents":10}"#;
    assert!(load_items(json).is_err());
}

#[test]
fn json_round_trip_preserves_items() {
    let items = load_items(SAMPLE).unwrap();
    let again = load_items(&to_json(&items).unwrap()).unwrap();
    assert_eq!(items, again);
}

```

```
#[test]
fn stock_value_overflow_is_none() {
    let item = Item {
        sku: "X".into(),
        name: "Bulk".into(),
        qty: 2,
        unit_cents: u64::MAX,
    };
    assert_eq!(stock_value(&[item]), None);
}
```

Data file: data/inventory.json

```
[
  {"sku": "ABC-0001", "name": "USB-C cable", "qty": 40, "unit_cents": 899},
  {"sku": "ABC-0002", "name": "Wall charger", "qty": 12, "unit_cents": 2499},
  {"sku": "ABC-0003", "name": "Phone case", "qty": 0, "unit_cents": 1299}
]
```

Expected result: cargo test reports 6 passed; stock value of data/inventory.json is 65,948 cents.

Verified before release: solution 6 tests passed; starter 0 passed / 6 failed as intended.

Stretch task: Add `#[serde(deny_unknown_fields)]` and a test proving an extra "colour" field is rejected.

Deliverables: your completed starter crate passing every gate, one extra boundary test, docs/review-record.md and docs/verification-record.md.

Topic 3: Rust Programming Controls, Functions and Features (Slides 96-136)

K3/A3 / LO3: Select essential programming controls and features to meet software design requirements (K3/A3).

if and match are expressions (Slides 97-99)

Expression: if and match produce a value you can bind with let.

Same type: Every branch must return the same type.

Ranges: 1..=4 matches 1, 2, 3 and 4 inclusive.

Wildcard: `_` matches everything else; use it deliberately.

```
let label = if qty == 0 { "out" } else { "in stock" };
```

```
let status = match qty {
    0 => "out of stock",
    1..=4 => "low",
    5..=99 => "ok",
    _ => "bulk",
};
```

Contract field	Exact rule
Boolean	Conditions must be bool; there is no truthy integer.
Exhaustive	match must cover every possible value.
Order	Arms are tried top to bottom; the first match wins.
Tests	Test each range boundary: 0, 1, 4, 5, 99, 100.

Common failure: error[E0004]: non-exhaustive patterns. What to do: Add the missing arm (the compiler lists it) instead of a blanket `_` that hides it.

How you know it works: Boundary tests cover every arm.

enum + match model every case (Slides 100-102)

Variants with data: Each variant carries exactly the data it needs.

Guards: if `q > stock` refines an arm with a condition.

Compiler help: Adding a variant makes every incomplete match fail to compile.

Errors as enums: `StockError` names each failure for callers and tests.

```
enum Movement { Receive(u32), Ship(u32), Adjust(i64) }

match movement {
  Movement::Receive(q) => stock.checked_add(q)
    .ok_or(StockError::Overflow),
  Movement::Ship(q) if q > stock => Err(Insufficient {...}),
  Movement::Ship(q) => Ok(stock - q),
  Movement::Adjust(d) => adjust(stock, d),
}
```

Contract field	Exact rule
Receive	checked_add; Overflow on u32 overflow.
Ship	Insufficient when requested > available.
Adjust	NegativeResult below zero; Overflow above u32.
Sequence	apply_all stops at the first error.

Common failure: New variant silently ignored. What to do: Avoid `_` in matches over your own enums so the compiler lists every place to update.

How you know it works: Activity 7: six tests cover every variant and every error.

Loops: for, while and loop (Slides 103-105)

`for`: Iterates over anything that implements `IntoIterator`.

`&items`: Borrows each element; `items` stays usable afterwards.

`enumerate`: Adds a 0-based index; add 1 for human line numbers.

`loop/break`: An infinite loop that can return a value with `break`.

```
for item in &items {                               // iterate by reference
  total += u64::from(item.qty);
}

for (index, line) in text.lines().enumerate() { /* .. */ }

let mut attempts = 0;
let code = loop {                                  // loop can return a value
  attempts += 1;
  if attempts == 3 { break "gave up"; }
};
```

Contract field	Exact rule
Preferred	for over iterators; no manual index bounds to get wrong.

Contract field	Exact rule
while	When the stop condition is not a collection end.
continue	Skip blank lines and headers early.
Early exit	? inside a loop returns the first error from the function.

Common failure: index out of bounds panic. What to do: Iterate with `for x in &v` or use `v.get(i)` which returns `Option`.

How you know it works: Activity 10 numbers errors from 1 and skips header and blank lines.

Structs and impl blocks (Slides 106-108)

Struct: Groups related data under one named type.

Private fields: Force every change through methods that keep the rules.

`&self` / `&mut self`: Methods declare whether they read or change the value.

derive: Debug, Clone, PartialEq, Default generated by the compiler.

```
#[derive(Debug, Default)]
pub struct Inventory {
    stock: HashMap<String, u32>,    // private field
}

impl Inventory {
    pub fn new() -> Self { Self::default() }
    pub fn quantity(&self, sku: &str) -> u32 { .. }
    pub fn ship(&mut self, sku: &str, qty: u32)
        -> Result<u32, String> { .. }
}
```

Contract field	Exact rule
Constructor	<code>new()</code> by convention; Default for empty values.
Readers	<code>&self</code> methods such as <code>quantity</code> and <code>low_stock</code> .
Writers	<code>&mut self</code> methods such as <code>receive</code> and <code>ship</code> .
Invariant	Stock never goes below zero because only <code>ship</code> can reduce it.

Common failure: cannot borrow as mutable (E0596). What to do: Declare the variable with `let mut`, or change the method to take `&self` if it only reads.

How you know it works: Activity 8: callers cannot touch the `HashMap` directly and six tests pass.

Collections: Vec and HashMap (Slides 109-111)

`Vec<T>`: Growable, ordered list; index or iterate.

`HashMap<K, V>`: Key lookup in constant average time; no order.

entry API: Insert-or-update with a single lookup.

`get` returns `Option`: A missing key is a normal case, not a crash.

```
let mut skus: Vec<String> = Vec::new();
skus.push("ABC-0001".to_string());
skus.sort();

let mut stock: HashMap<String, u32> = HashMap::new();
*stock.entry("ABC-0001".into()).or_insert(0) += 5;
let level = stock.get("ABC-0001").copied().unwrap_or(0);
```

```
if let Some(q) = stock.get_mut("ABC-0001") { *q -= 1; }
```

Contract field	Exact rule
Order	Sort before output because HashMap order is random.
Ownership	Keys are owned Strings; look up with &str.
Missing key	unwrap_or(0) or an explicit error, never map[key] panic.
Choice	Vec for sequences; HashMap for lookup by id.

Common failure: Test passes locally, fails elsewhere. What to do: Output depended on HashMap order; sort the result before comparing.

How you know it works: low_stock returns a sorted Vec so tests are deterministic.

Traits and generics share behaviour (Slides 112-114)

Trait: A named set of methods a type promises to provide.

Default method: Written once, inherited by every implementor.

Generic T: Priced: One type per call, resolved at compile time (fast).

&dyn Priced: Different types in one list, resolved at run time.

```
pub trait Priced {
    fn unit_cents(&self) -> u64;
    fn quantity(&self) -> u32;
    fn line_cents(&self) -> Option<u64> {          // default
        self.unit_cents().checked_mul(u64::from(self.quantity()))
    }
}

impl Priced for Product { .. }
impl Priced for Service { .. }
```

```
fn order_total<T: Priced>(lines: &[T]) -> Option<u64>
fn mixed_total(lines: &[&dyn Priced]) -> Option<u64>
```

Contract field	Exact rule
Contract	Implementors supply unit_cents and quantity only.
Extension	A new type joins by implementing the trait; totals are unchanged.
Dispatch	Generics for speed; trait objects for mixed collections.
Tests	Test the default method once via any implementor.

Common failure: the trait bound `X: Priced` is not satisfied. What to do: Implement Priced for X, or pass a type that already implements it.

How you know it works: Activity 9 totals products, services and a mixed order.

Closures and iterator chains (Slides 115-117)

Closure: |p| ... is an anonymous function that can capture variables.

Lazy: filter and map do nothing until collect or a fold consumes them.

collect: The target type (Vec, String, Option<Vec>) decides the result.

try_fold: A fold that stops at the first None or Err.

```
let expensive: Vec<&str> = products
    .iter()
    .filter(|p| p.unit_cents >= 1_000)
    .map(|p| p.name.as_str())
    .collect();

let total = lines.iter().try_fold(0u64, |acc, line| {
    acc.checked_add(line.line_cents())?
});
```

Contract field	Exact rule
Readability	Each step names one transformation.
Safety	No manual indexes, so no out-of-bounds errors.
Performance	Compiles to the same machine code as a hand loop.
Order	Iterator chains keep input order unless you sort.

Common failure: value of type Vec<&str> cannot be built from iterator. What to do: Check what map returns (&str vs String) and annotate the collect target type.

How you know it works: names_at_least keeps input order; try_fold returns None on overflow.

Option and Result are control features (Slides 118-120)

Option<T>: Some(value) or None: absence without null.

Result<T, E>: Ok(value) or Err(reason): failure without exceptions.

Combinators: map, and_then, ok_or, unwrap_or transform without match.

?: Returns early with the None/Err, otherwise unwraps the value.

```
fn find(sku: &str) -> Option<u32> // may be absent
fn ship(sku: &str, q: u32) -> Result<u32, String> // may fail
```

```
match find("ABC-0001") {
    Some(q) => println!("{q} on hand"),
    None => println!("unknown SKU"),
}
let q = find("ABC-0001").unwrap_or(0);
let left = ship("ABC-0001", 2)?; // propagate
```

Contract field	Exact rule
Absent	Use Option when there is no reason to report.
Failed	Use Result when the caller needs to know why.
Library code	Return Option/Result; let the caller decide.
Binaries	Convert errors to messages and exit codes at the edge.

Common failure: the ? operator can only be used in a function that returns Result or Option. What to do: Change the function return type, or handle the value with match.

How you know it works: No unwrap() on input data remains in any solution crate.

Activity 7: Apply stock movements with enums and match (Slides 121-125)

Folder: labs/activity-07-match-control-flow. Maps to K3/A3. Suggested time: 30 minutes.

Goal and scenario: Stock changes through receipts, shipments and manual adjustments. An enum names every movement and match forces each one to be handled, including the failure cases.

Exact contract: `Movement::{Receive(u32), Ship(u32), Adjust(i64)}. apply(stock, movement) -> Result<u32, StockError>`: shipping more than available is `Insufficient`; results below zero are `NegativeResult`; above `u32` are `Overflow`. `apply_all` stops at the first error.

Step-by-step: Activity 7

1. Read the specification. Read the Goal and Exact contract sections below and the files in `data/` (if any). Write down each input, its type and valid range, the failure behaviour and the boundary values before you open any code.
2. Open the activity in VS Code. Open this activity folder in Visual Studio Code with the `rust-analyzer` extension enabled. Confirm the toolchain with `rustc --version` and `cargo --version` in the integrated terminal.
3. Run the reference solution. In the terminal run: `cd solution && cargo test`. Confirm the result line shows every test passing. This proves your toolchain works and shows the target behaviour. (`cargo test` reports 6 passed, covering every variant and every error.)
4. Run the starter and read the failures. Run: `cd ../starter && cargo test`. Every test that calls an unfinished function fails with "not yet implemented" from `todo!()`. This is expected: the tests are the specification you will implement against.
5. Plan with the AI assistant. Paste the "Plan before code" prompt from `PROMPTS.md` into your AI coding assistant (for example GitHub Copilot Chat, Claude or ChatGPT). Compare its edge cases with your own list from step 1 and record agreements and disagreements in `docs/review-record.md`.
6. Vibe-code the implementation. Use the "Generate with AI" prompt to generate code for `starter/src/lib.rs`. Review the proposal before accepting it: public signatures unchanged, no `unsafe`, no `unwrap()` or `expect()` on input data, no new crates, and every line explained. Reject or edit anything you cannot justify.
7. Test until green. Run `cargo test` after each accepted change. If a test fails or the code does not compile, use the "Repair from evidence" prompt with the exact cargo output. Never edit expected values in tests to make them pass.
8. Apply the quality gates. Run every gate from the starter folder: `cargo test ; cargo clippy --all-targets -- -D warnings ; cargo fmt --check`. Fix each clippy warning and formatting difference; these are the organisation's coding standards.
9. Add your own boundary test. Predict one more boundary case from the specification and add it to the test file in `starter/tests/`. Run it against both starter and solution. Stretch: Add `Movement::Return(u32)` and let the compiler show every match you must update.
10. Record the evidence. Complete `docs/verification-record.md` with the commands you ran, the exact result lines (for example "test result: ok. 5 passed"), the AI proposals you accepted or rejected and why. Compare your code with solution/ only after your own tests pass.

Commands: Activity 7

```
cd solution && cargo test
cd ../starter
cargo test
cargo clippy --all-targets -- -D warnings
cargo fmt --check
```

Vibe-coding prompts: Activity 7

Plan before code: Act as a senior Rust reviewer. Specification: `Movement::{Receive(u32), Ship(u32), Adjust(i64)}. apply(stock, movement) -> Result<u32, StockError>`: shipping more than available is `Insufficient`; results below zero are `NegativeResult`; above `u32` are `Overflow`. `apply_all` stops at the

first error. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI: Implement "Apply stock movements with enums and match" in starter/src so that the supplied tests pass. Use enum + match with exhaustive arms; no wildcard `_` arm that hides future variants. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence: Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

The same prompts are in labs/activity-07-match-control-flow/PROMPTS.md and Vibe-Coding-Prompts.pdf.

Reference solution: Activity 7

solution/src/lib.rs

```
//! Activity 7: control flow with enums, match and early return.

/// One stock movement.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Movement {
    /// Units received from a supplier.
    Receive(u32),
    /// Units shipped to a customer.
    Ship(u32),
    /// Signed stock-take correction.
    Adjust(i64),
}

/// Why a movement was refused.
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum StockError {
    /// A shipment asked for more than is available.
    Insufficient {
        /// Units on hand.
        available: u32,
        /// Units requested.
        requested: u32,
    },
    /// The result would exceed u32::MAX.
    Overflow,
    /// The result would be below zero.
    NegativeResult,
}

/// Applies one movement and returns the new stock level.
pub fn apply(stock: u32, movement: Movement) -> Result<u32, StockError> {
    match movement {
        Movement::Receive(qty) => {
            stock.checked_add(qty).ok_or(StockError::Overflow)
        }
        Movement::Ship(qty) if qty > stock => Err(StockError::Insufficient {
            available: stock,
            requested: qty,
        }),
        Movement::Ship(qty) => Ok(stock - qty),
        Movement::Adjust(delta) => {
            let next = i64::from(stock)
                .checked_add(delta)
                .ok_or(StockError::Overflow)?;
            if next < 0 {
```

```

        return Err(StockError::NegativeResult);
    }
    u32::try_from(next).map_err(|_| StockError::Overflow)
}
}

/// Applies movements in order and stops at the first error.
pub fn apply_all(
    stock: u32,
    movements: &[Movement],
) -> Result<u32, StockError> {
    let mut current = stock;
    for movement in movements {
        current = apply(current, *movement)?;
    }
    Ok(current)
}

```

Acceptance tests: Activity 7

tests/acceptance.rs (identical in starter/ and solution/)

```

use activity07::{apply, apply_all, Movement, StockError};

#[test]
fn receive_and_ship() {
    assert_eq!(apply(10, Movement::Receive(5)), Ok(15));
    assert_eq!(apply(10, Movement::Ship(10)), Ok(0));
}

#[test]
fn shipping_too_much_is_insufficient() {
    assert_eq!(
        apply(3, Movement::Ship(5)),
        Err(StockError::Insufficient {
            available: 3,
            requested: 5
        })
    );
}

#[test]
fn adjustments_are_bounded_both_ways() {
    assert_eq!(apply(10, Movement::Adjust(-4)), Ok(6));
    assert_eq!(
        apply(3, Movement::Adjust(-4)),
        Err(StockError::NegativeResult)
    );
    assert_eq!(
        apply(u32::MAX, Movement::Adjust(1)),
        Err(StockError::Overflow)
    );
    assert_eq!(
        apply(1, Movement::Adjust(i64::MAX)),
        Err(StockError::Overflow)
    );
}

#[test]
fn receive_overflow_is_reported() {
    assert_eq!(
        apply(u32::MAX, Movement::Receive(1)),
        Err(StockError::Overflow)
    );
}

```

```

}

#[test]
fn apply_all_runs_in_order() {
    let moves = [
        Movement::Receive(10),
        Movement::Ship(4),
        Movement::Adjust(-1),
    ];
    assert_eq!(apply_all(0, &moves), Ok(5));
}

#[test]
fn apply_all_stops_at_the_first_error() {
    let moves = [
        Movement::Receive(2),
        Movement::Ship(5),
        Movement::Receive(100),
    ];
    assert_eq!(
        apply_all(0, &moves),
        Err(StockError::Insufficient {
            available: 2,
            requested: 5
        })
    );
}

```

Expected result: cargo test reports 6 passed, covering every variant and every error.

Verified before release: solution 6 tests passed; starter 0 passed / 6 failed as intended.

Stretch task: Add `Movement::Return(u32)` and let the compiler show every match you must update.

Deliverables: your completed starter crate passing every gate, one extra boundary test, docs/review-record.md and docs/verification-record.md.

Activity 8: Track stock in a HashMap (Slides 126-130)

Folder: labs/activity-08-hashmap-inventory. Maps to K3/A3. Suggested time: 30 minutes.

Goal and scenario: StockPilot needs fast lookup by SKU. You wrap a HashMap in an Inventory type so every change goes through methods that enforce the stock rules.

Exact contract: `Inventory::receive(&mut self, sku, qty) -> Option<u32>` (None on overflow, unchanged); `ship -> Result<u32, String>` (unknown SKU or insufficient stock is Err, unchanged); `quantity(sku) -> u32` (0 when unknown); `low_stock(threshold) -> Vec<(String, u32)>` strictly below threshold, sorted by SKU.

Step-by-step: Activity 8

1. Read the specification. Read the Goal and Exact contract sections below and the files in data/ (if any). Write down each input, its type and valid range, the failure behaviour and the boundary values before you open any code.
2. Open the activity in VS Code. Open this activity folder in Visual Studio Code with the rust-analyzer extension enabled. Confirm the toolchain with `rustc --version` and `cargo --version` in the integrated terminal.
3. Run the reference solution. In the terminal run: `cd solution && cargo test`. Confirm the result line shows every test passing. This proves your toolchain works and shows the target behaviour. (cargo test reports 6 passed; `low_stock(5)` returns AAA-0001 and BBB-0002 only.)

4. Run the starter and read the failures. Run: `cd ../starter && cargo test`. Every test that calls an unfinished function fails with "not yet implemented" from `todo!()`. This is expected: the tests are the specification you will implement against.
5. Plan with the AI assistant. Paste the "Plan before code" prompt from `PROMPTS.md` into your AI coding assistant (for example GitHub Copilot Chat, Claude or ChatGPT). Compare its edge cases with your own list from step 1 and record agreements and disagreements in `docs/review-record.md`.
6. Vibe-code the implementation. Use the "Generate with AI" prompt to generate code for `starter/src/lib.rs`. Review the proposal before accepting it: public signatures unchanged, no `unsafe`, no `unwrap()` or `expect()` on input data, no new crates, and every line explained. Reject or edit anything you cannot justify.
7. Test until green. Run `cargo test` after each accepted change. If a test fails or the code does not compile, use the "Repair from evidence" prompt with the exact cargo output. Never edit expected values in tests to make them pass.
8. Apply the quality gates. Run every gate from the starter folder: `cargo test ; cargo clippy --all-targets -- -D warnings ; cargo fmt --check`. Fix each clippy warning and formatting difference; these are the organisation's coding standards.
9. Add your own boundary test. Predict one more boundary case from the specification and add it to the test file in `starter/tests/`. Run it against both starter and solution. Stretch: Add a `remove_sku` method that refuses to delete an SKU with stock on hand.
10. Record the evidence. Complete `docs/verification-record.md` with the commands you ran, the exact result lines (for example "test result: ok. 5 passed"), the AI proposals you accepted or rejected and why. Compare your code with solution/ only after your own tests pass.

Commands: Activity 8

```
cd solution && cargo test
cd ../starter
cargo test
cargo clippy --all-targets -- -D warnings
cargo fmt --check
```

Vibe-coding prompts: Activity 8

Plan before code: Act as a senior Rust reviewer. Specification: `Inventory::receive(&mut self, sku, qty)` -> `Option<u32>` (None on overflow, unchanged); `ship` -> `Result<u32, String>` (unknown SKU or insufficient stock is `Err`, unchanged); `quantity(sku)` -> `u32` (0 when unknown); `low_stock(threshold)` -> `Vec<(String, u32)>` strictly below threshold, sorted by SKU. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI: Implement "Track stock in a HashMap" in `starter/src` so that the supplied tests pass. Keep the `HashMap` private and the method signatures unchanged; no `unwrap()` on lookups. Keep public signatures unchanged, use no `unsafe` code, and explain each design decision in one sentence.

Repair from evidence: Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

The same prompts are in `labs/activity-08-hashmap-inventory/PROMPTS.md` and `Vibe-Coding-Prompts.pdf`.

Reference solution: Activity 8

solution/src/lib.rs

```
//! Activity 8: an inventory type wrapping a HashMap.

use std::collections::HashMap;

/// Stock levels keyed by SKU.
#[derive(Debug, Default)]
pub struct Inventory {
    stock: HashMap<String, u32>,
}

impl Inventory {
    /// Creates an empty inventory.
    pub fn new() -> Self {
        Self::default()
    }

    /// Adds units; returns the new level, or `None` on overflow.
    pub fn receive(&mut self, sku: &str, qty: u32) -> Option<u32> {
        let level = self.stock.entry(sku.to_string()).or_insert(0);
        *level = level.checked_add(qty)?;
        Some(*level)
    }

    /// Removes units; unknown SKUs and shortages are errors.
    pub fn ship(&mut self, sku: &str, qty: u32) -> Result<u32, String> {
        let level = self
            .stock
            .get_mut(sku)
            .ok_or_else(|| format!("unknown SKU {sku}"))?;
        if qty > *level {
            return Err(format!("only {level} of {sku} available"));
        }
        *level -= qty;
        Ok(*level)
    }

    /// Units on hand; zero for an unknown SKU.
    pub fn quantity(&self, sku: &str) -> u32 {
        self.stock.get(sku).copied().unwrap_or(0)
    }

    /// SKUs strictly below `threshold`, sorted by SKU.
    pub fn low_stock(&self, threshold: u32) -> Vec<(String, u32)> {
        let mut low: Vec<(String, u32)> = self
            .stock
            .iter()
            .filter(|(&_, qty)| **qty < threshold)
            .map(|(sku, qty)| (sku.clone(), *qty))
            .collect();
        low.sort();
        low
    }
}
```

Acceptance tests: Activity 8

tests/acceptance.rs (identical in starter/ and solution/)

```
use activity08::Inventory;

#[test]
```

```

fn unknown_sku_has_zero_quantity() {
    assert_eq!(Inventory::new().quantity("ABC-0001"), 0);
}

#[test]
fn receive_accumulates() {
    let mut inv = Inventory::new();
    assert_eq!(inv.receive("ABC-0001", 5), Some(5));
    assert_eq!(inv.receive("ABC-0001", 3), Some(8));
    assert_eq!(inv.quantity("ABC-0001"), 8);
}

#[test]
fn receive_overflow_leaves_stock_unchanged() {
    let mut inv = Inventory::new();
    inv.receive("ABC-0001", u32::MAX);
    assert_eq!(inv.receive("ABC-0001", 1), None);
    assert_eq!(inv.quantity("ABC-0001"), u32::MAX);
}

#[test]
fn ship_reduces_stock() {
    let mut inv = Inventory::new();
    inv.receive("ABC-0001", 5);
    assert_eq!(inv.ship("ABC-0001", 2), Ok(3));
}

#[test]
fn ship_errors_leave_stock_unchanged() {
    let mut inv = Inventory::new();
    inv.receive("ABC-0001", 5);
    assert!(inv.ship("ZZZ-9999", 1).unwrap_err().contains("unknown SKU"));
    assert!(inv.ship("ABC-0001", 6).is_err());
    assert_eq!(inv.quantity("ABC-0001"), 5);
}

#[test]
fn low_stock_is_strict_and_sorted() {
    let mut inv = Inventory::new();
    inv.receive("CCC-0003", 5);
    inv.receive("BBB-0002", 3);
    inv.receive("AAA-0001", 0);
    assert_eq!(
        inv.low_stock(5),
        vec!["AAA-0001".to_string(), 0], ("BBB-0002".to_string(), 3)
    );
}

```

Expected result: cargo test reports 6 passed; low_stock(5) returns AAA-0001 and BBB-0002 only.

Verified before release: solution 6 tests passed; starter 0 passed / 6 failed as intended.

Stretch task: Add a remove_sku method that refuses to delete an SKU with stock on hand.

Deliverables: your completed starter crate passing every gate, one extra boundary test, docs/review-record.md and docs/verification-record.md.

Activity 9: Price mixed order lines with traits and iterators (Slides 131-135)

Folder: labs/activity-09-traits-iterators. Maps to K3/A3. Suggested time: 30 minutes.

Goal and scenario: Orders can contain products and services (for example, device set-up hours). A Priced trait gives both one pricing contract, and iterator chains total and filter them without manual index loops.

Exact contract: trait Priced { unit_cents, quantity, line_cents (default, checked) }. order_total<T: Priced>(&[T]) and mixed_total(&[&dyn Priced]) return Option<u64> (None on overflow; empty is Some(0)). names_at_least(&[Product], min_cents) -> Vec<&str> in input order.

Step-by-step: Activity 9

1. Read the specification. Read the Goal and Exact contract sections below and the files in data/ (if any). Write down each input, its type and valid range, the failure behaviour and the boundary values before you open any code.
2. Open the activity in VS Code. Open this activity folder in Visual Studio Code with the rust-analyzer extension enabled. Confirm the toolchain with rustc --version and cargo --version in the integrated terminal.
3. Run the reference solution. In the terminal run: cd solution && cargo test. Confirm the result line shows every test passing. This proves your toolchain works and shows the target behaviour. (cargo test reports 5 passed; the mixed order totals 14,297 cents.)
4. Run the starter and read the failures. Run: cd ../starter && cargo test. Every test that calls an unfinished function fails with "not yet implemented" from todo!(). This is expected: the tests are the specification you will implement against.
5. Plan with the AI assistant. Paste the "Plan before code" prompt from PROMPTS.md into your AI coding assistant (for example GitHub Copilot Chat, Claude or ChatGPT). Compare its edge cases with your own list from step 1 and record agreements and disagreements in docs/review-record.md.
6. Vibe-code the implementation. Use the "Generate with AI" prompt to generate code for starter/src/lib.rs. Review the proposal before accepting it: public signatures unchanged, no unsafe, no unwrap() or expect() on input data, no new crates, and every line explained. Reject or edit anything you cannot justify.
7. Test until green. Run cargo test after each accepted change. If a test fails or the code does not compile, use the "Repair from evidence" prompt with the exact cargo output. Never edit expected values in tests to make them pass.
8. Apply the quality gates. Run every gate from the starter folder: cargo test ; cargo clippy --all-targets -- -D warnings ; cargo fmt --check. Fix each clippy warning and formatting difference; these are the organisation's coding standards.
9. Add your own boundary test. Predict one more boundary case from the specification and add it to the test file in starter/tests/. Run it against both starter and solution. Stretch: Add a Subscription type implementing Priced and include it in mixed_total without changing that function.
10. Record the evidence. Complete docs/verification-record.md with the commands you ran, the exact result lines (for example "test result: ok. 5 passed"), the AI proposals you accepted or rejected and why. Compare your code with solution/ only after your own tests pass.

Commands: Activity 9

```
cd solution && cargo test
cd ../starter
cargo test
cargo clippy --all-targets -- -D warnings
cargo fmt --check
```

Vibe-coding prompts: Activity 9

Plan before code: Act as a senior Rust reviewer. Specification: trait Priced { unit_cents, quantity, line_cents (default, checked) }. order_total<T: Priced>(&[T]) and mixed_total(&[&dyn Priced]) return Option<u64> (None on overflow; empty is Some(0)). names_at_least(&[Product], min_cents) ->

Vec<str> in input order. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI: Implement "Price mixed order lines with traits and iterators" in starter/src so that the supplied tests pass. Implement with iterator adaptors (iter, filter, map, try_fold); no index-based for loops. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence: Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

The same prompts are in labs/activity-09-traits-iterators/PROMPTS.md and Vibe-Coding-Prompts.pdf.

Reference solution: Activity 9

solution/src/lib.rs

```
//! Activity 9: traits, generics, trait objects and iterator chains.

/// Anything that can be priced as quantity x unit price.
pub trait Priced {
    /// Price of one unit in cents.
    fn unit_cents(&self) -> u64;
    /// Number of units.
    fn quantity(&self) -> u32;
    /// Line value in cents; `None` on overflow. Shared by every implementor.
    fn line_cents(&self) -> Option<u64> {
        self.unit_cents().checked_mul(u64::from(self.quantity()))
    }
}

/// A physical product line.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Product {
    /// Display name.
    pub name: String,
    /// Unit price in cents.
    pub unit_cents: u64,
    /// Units ordered.
    pub qty: u32,
}

/// A billable service line.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Service {
    /// What is being done.
    pub description: String,
    /// Hourly rate in cents.
    pub rate_cents: u64,
    /// Hours billed.
    pub hours: u32,
}

impl Priced for Product {
    fn unit_cents(&self) -> u64 {
        self.unit_cents
    }
    fn quantity(&self) -> u32 {
        self.qty
    }
}
```

```

impl Priced for Service {
    fn unit_cents(&self) -> u64 {
        self.rate_cents
    }
    fn quantity(&self) -> u32 {
        self.hours
    }
}

/// Totals lines of one type (static dispatch).
pub fn order_total<T: Priced>(lines: &[T]) -> Option<u64> {
    lines
        .iter()
        .try_fold(0u64, |total, line| total.checked_add(line.line_cents()?))
}

/// Totals lines of different types (dynamic dispatch).
pub fn mixed_total(lines: &[&dyn Priced]) -> Option<u64> {
    lines
        .iter()
        .try_fold(0u64, |total, line| total.checked_add(line.line_cents()?))
}

/// Names of products priced at or above `min_cents`, in input order.
pub fn names_at_least(products: &[Product], min_cents: u64) -> Vec<&str> {
    products
        .iter()
        .filter(|product| product.unit_cents >= min_cents)
        .map(|product| product.name.as_str())
        .collect()
}

```

Acceptance tests: Activity 9

tests/acceptance.rs (identical in starter/ and solution/)

```

use activity09::{
    mixed_total, names_at_least, order_total, Priced, Product, Service,
};

fn products() -> Vec<Product> {
    vec![
        Product {
            name: "Cable".into(),
            unit_cents: 899,
            qty: 2,
        },
        Product {
            name: "Charger".into(),
            unit_cents: 2499,
            qty: 1,
        },
    ]
}

fn setup() -> Service {
    Service {
        description: "Device set-up".into(),
        rate_cents: 5000,
        hours: 2,
    }
}

```

```

#[test]
fn default_method_prices_a_line() {
    assert_eq!(setup().line_cents(), Some(10_000));
}

#[test]
fn generic_total_for_one_type() {
    assert_eq!(order_total(&products()), Some(4_297));
    assert_eq!(order_total::<Product>(&[]), Some(0));
}

#[test]
fn trait_objects_mix_types() {
    let items = products();
    let service = setup();
    let lines: Vec<&dyn Priced> = vec! [&items[0], &items[1], &service];
    assert_eq!(mixed_total(&lines), Some(14_297));
}

#[test]
fn overflow_is_none() {
    let bulk = Product {
        name: "Bulk".into(),
        unit_cents: u64::MAX,
        qty: 2,
    };
    assert_eq!(order_total(&[bulk]), None);
}

#[test]
fn filter_and_map_keep_input_order() {
    assert_eq!(names_at_least(&products(), 1_000), ["Charger"]);
    assert_eq!(names_at_least(&products(), 0), ["Cable", "Charger"]);
}

```

Expected result: cargo test reports 5 passed; the mixed order totals 14,297 cents.

Verified before release: solution 5 tests passed; starter 1 passed / 4 failed as intended.

Stretch task: Add a Subscription type implementing Priced and include it in mixed_total without changing that function.

Deliverables: your completed starter crate passing every gate, one extra boundary test, docs/review-record.md and docs/verification-record.md.

Topic 4: Testing, Debugging and Integrating Rust Components (Slides 137-174)

K4/A4 / LO4: Examine the interoperability and functionality of programming components (K4/A4).

Unit tests live beside the code (Slides 138-140)

#[test]: Marks a function that cargo test runs.

#[cfg(test)]: Compiled only for tests; not shipped in the binary.

use super::*: Unit tests can reach private items of their module.

assert_eq!: Shows left and right values when it fails.

```

pub fn needs_reorder(on_hand: u32, point: u32) -> bool {
    on_hand <= point
}

```

```
#[cfg(test)]
mod tests {
    use super::*;
    #[test]
    fn boundary_is_inclusive() {
        assert!(needs_reorder(80, 80));
        assert!(!needs_reorder(81, 80));
    }
}
```

Contract field	Exact rule
Name	Describe the rule: boundary_is_inclusive, not test1.
One behaviour	Each test checks one rule so failures are specific.
Boundaries	Test at, just below and just above every limit.
Speed	Unit tests run in milliseconds; run them constantly.

Common failure: Test passes for the wrong reason. What to do: Predict the expected value from the specification before running, never copy actual output.

How you know it works: cargo test lists each test by name with ok.

Integration tests check components together (Slides 141-143)

tests/ folder: Each file is compiled like an external user of the crate.

Fixtures: include_str! embeds a data file at compile time.

End to end: Parse, calculate and report are checked together.

Public API: If a test needs a private item, the API may be incomplete.

```
tests/integration.rs      # separate crate, public API only
```

```
use activity11::{parse_lines, summarise, Summary};
const SAMPLE: &str = include_str!("../data/stock.csv");
```

```
#[test]
fn end_to_end_summary() {
    assert_eq!(summarise(SAMPLE, 5), Ok(Summary {
        lines: 4, total_cents: 52_655,
        low_stock: vec!["ABC-0003".into()] }));
}
```

Contract field	Exact rule
Unit	One function, fast, may use private items.
Integration	Several components through the public API.
Process	Run the real binary (Activity 12).
Data	Fixture files live in data/ and are versioned.

Common failure: Unit tests pass but the summary is wrong. What to do: Components disagree; add an integration test with a known fixture total.

How you know it works: Activity 11: the end-to-end test fixes the expected total at 52,655 cents.

Custom errors and the ? operator (Slides 144-146)

Error enum: Each failure is a named, testable variant.

Display: Human-readable message for logs and operators.

Error trait: Lets the error work with Box<dyn Error> and ?.

?: Stops at the first bad field and returns its error.

```
#[derive(Debug, PartialEq)]
pub enum ParseError {
    FieldCount(usize),
    EmptySku,
    BadNumber { field: &'static str, value: String },
}
impl fmt::Display for ParseError { .. }
impl std::error::Error for ParseError {}

let qty = number("qty", fields[1])?; // early return on Err
```

Contract field	Exact rule
Recoverable	Bad input is Result, not panic.
Context	Include the field and the offending value.
Aggregation	parse_all keeps good lines and numbered errors.
Tests	assert_eq! on the exact error variant and message.

Common failure: Error says only "invalid digit". What to do: Map low-level errors into your enum with field name and value via map_err.

How you know it works: Activity 10 reports errors on lines 3, 4 and 6 with readable messages.

Read a failing test like a detective (Slides 147-149)

Test name: Tells you which rule is broken.

left / right: Actual value versus expected value.

Arithmetic: The difference often identifies the missing or extra data.

Location: File and line of the failing assertion.

```
---- total_includes_the_first_line stdout ----
thread 'total_includes_the_first_line' panicked at
tests/integration.rs:13:5:
assertion `left == right` failed
  left: Some(16695)
 right: Some(52655)

52655 - 16695 = 35960 = 40 x 899 -> first line missing
```

Contract field	Exact rule
Reproduce	Run one test: cargo test total_includes
Backtrace	RUST_BACKTRACE=1 cargo test shows the call path.
Isolate	Check each component with its own test.
Fix	Change the code, never the expected value.

Common failure: Changing the expected value to make it pass. What to do: Forbidden: the expected value comes from the specification; find the defect instead.

How you know it works: After the fix, the same test passes and stays as a regression test.

Debug with dbg!, focused tests and AI (Slides 150-152)

dbg!: Prints the expression, its value and where it ran; returns the value.

Filter: cargo test <name> runs only matching tests.

--nocapture: Shows println!/dbg! output from passing tests too.

AI: Ask for a diagnosis first, then a minimal patch.

```
let total = dbg!(total_cents(&lines));    // prints file:line + value
```

```
cargo test low_stock -- --nocapture      # one test, show output
RUST_BACKTRACE=1 cargo test
```

Prompt: "Here is the failing test and total_cents().
Explain why the actual value is 16695, point to the line,
and propose the smallest fix. Do not change the test."

Contract field	Exact rule
Hypothesis	State what you think is wrong before changing code.
One change	Change one thing, rerun, observe.
Cleanup	Remove dbg! before committing; clippy can flag it.
Record	Symptom, cause, fix and test in the review record.

Common failure: Fix works for the test but breaks another. What to do: Run the whole suite after every change, not only the failing test.

How you know it works: Activity 11 ends with 6 passed and a written defect record.

Interoperability: arguments, files, exit codes (Slides 153-155)

Arguments: std::env::args(); validate count and types.

Files: std::fs::read_to_string returns Result.

stdout/stderr: Data to stdout, diagnostics to stderr.

ExitCode: main returns ExitCode so scripts can react.

```
$ stockpilot data/stock.csv 13
ITEMS 4
TOTAL $526.55
LOW STOCK ABC-0002, ABC-0003, ABC-0004
$ echo $?          # 0
```

```
$ stockpilot          # usage error -> stderr, exit 2
$ stockpilot nope.csv # cannot read -> stderr, exit 1
```

Contract field	Exact rule
0	Success: report printed.
1	Data problem: unreadable file or malformed row.
2	Usage problem: wrong arguments.
Tests	Process tests run the binary via CARGO_BIN_EXE_<name>.

Common failure: Script cannot tell success from failure. What to do: Return a non-zero ExitCode and print the error to stderr.

How you know it works: Activity 12 process tests check stdout, stderr and exit codes 0, 1 and 2.

Serde: typed data at the boundary (Slides 156-158)

Derive: Serialize/Deserialize are generated from the struct.

Validation: Types reject wrong shapes before business logic runs.

Round trip: Serialise then parse must give equal values.

Errors: `serde_json::Error` reports line, column and cause.

```
#[derive(Serialize, Deserialize, Debug, PartialEq)]
pub struct Item {
    pub sku: String,
    pub name: String,
    pub qty: u32,           // -1 in JSON -> error
    pub unit_cents: u64,
}
```

```
let items: Vec<Item> = serde_json::from_str(json)?;
let text = serde_json::to_string_pretty(&items)?;
```

Contract field	Exact rule
Missing field	Error unless <code>#[serde(default)]</code> is chosen.
Unknown field	Ignored unless <code>#[serde(deny_unknown_fields)]</code> .
Numbers	Out-of-range or negative values for <code>u32</code> are errors.
Contract	The struct is the documented JSON schema.

Common failure: invalid value: integer -1, expected u32. What to do: Correct the source data; do not widen the type to accept impossible values.

How you know it works: Activity 6 rejects a missing field and a negative quantity.

Activity 10: Parse CSV records with Result and custom errors (Slides 159-163)

Folder: `labs/activity-10-error-handling`. Maps to K4/A4. Suggested time: 30 minutes.

Goal and scenario: Stock files arrive with typos. Instead of crashing, StockPilot must report exactly which line is wrong and why, while still importing every valid line.

Exact contract: `parse_record(&str) -> Result<Record, ParseError>` with `ParseError::(FieldCount(n), EmptySku, BadNumber{field, value})` and a Display message for each. `parse_all(&str)` skips blank lines and the header, returning valid records and (1-based line number, error) pairs.

Step-by-step: Activity 10

1. Read the specification. Read the Goal and Exact contract sections below and the files in `data/` (if any). Write down each input, its type and valid range, the failure behaviour and the boundary values before you open any code.
2. Open the activity in VS Code. Open this activity folder in Visual Studio Code with the `rust-analyzer` extension enabled. Confirm the toolchain with `rustc --version` and `cargo --version` in the integrated terminal.
3. Run the reference solution. In the terminal run: `cd solution && cargo test`. Confirm the result line shows every test passing. This proves your toolchain works and shows the target behaviour. (`cargo test` reports 5 passed; `data/stock.csv` yields 2 records and errors on lines 3, 4 and 6.)

4. Run the starter and read the failures. Run: `cd ../starter && cargo test`. Every test that calls an unfinished function fails with "not yet implemented" from `todo!()`. This is expected: the tests are the specification you will implement against.
5. Plan with the AI assistant. Paste the "Plan before code" prompt from `PROMPTS.md` into your AI coding assistant (for example GitHub Copilot Chat, Claude or ChatGPT). Compare its edge cases with your own list from step 1 and record agreements and disagreements in `docs/review-record.md`.
6. Vibe-code the implementation. Use the "Generate with AI" prompt to generate code for `starter/src/lib.rs`. Review the proposal before accepting it: public signatures unchanged, no `unsafe`, no `unwrap()` or `expect()` on input data, no new crates, and every line explained. Reject or edit anything you cannot justify.
7. Test until green. Run `cargo test` after each accepted change. If a test fails or the code does not compile, use the "Repair from evidence" prompt with the exact cargo output. Never edit expected values in tests to make them pass.
8. Apply the quality gates. Run every gate from the starter folder: `cargo test ; cargo clippy --all-targets -- -D warnings ; cargo fmt --check`. Fix each clippy warning and formatting difference; these are the organisation's coding standards.
9. Add your own boundary test. Predict one more boundary case from the specification and add it to the test file in `starter/tests/`. Run it against both starter and solution. Stretch: Add `ParseError::NegativeValue` for "-1" so the message is clearer than `BadNumber`.
10. Record the evidence. Complete `docs/verification-record.md` with the commands you ran, the exact result lines (for example "test result: ok. 5 passed"), the AI proposals you accepted or rejected and why. Compare your code with solution/ only after your own tests pass.

Commands: Activity 10

```
cd solution && cargo test
cd ../starter
cargo test
cargo clippy --all-targets -- -D warnings
cargo fmt --check
```

Vibe-coding prompts: Activity 10

Plan before code: Act as a senior Rust reviewer. Specification: `parse_record(&str) -> Result<Record, ParseError>` with `ParseError::{FieldCount(n), EmptySku, BadNumber{field, value}}` and a Display message for each. `parse_all(&str)` skips blank lines and the header, returning valid records and (1-based line number, error) pairs. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI: Implement "Parse CSV records with Result and custom errors" in `starter/src` so that the supplied tests pass. Return `Result` everywhere; no `unwrap()`, `expect()` or `panic!` in library code. Keep public signatures unchanged, use no `unsafe` code, and explain each design decision in one sentence.

Repair from evidence: Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

The same prompts are in `labs/activity-10-error-handling/PROMPTS.md` and `Vibe-Coding-Prompts.pdf`.

Reference solution: Activity 10

`solution/src/lib.rs`

```
///! Activity 10: recoverable errors with Result, ? and a custom error type.
```

```
use std::fmt;
```

```
/// One valid stock record.
```

```
#[derive(Debug, Clone, PartialEq, Eq)]
```

```
pub struct Record {  
    /// Product code.  
    pub sku: String,  
    /// Units on hand.  
    pub qty: u32,  
    /// Unit cost in cents.  
    pub unit_cents: u64,  
}
```

```
/// Why a line could not be parsed.
```

```
#[derive(Debug, Clone, PartialEq, Eq)]
```

```
pub enum ParseError {  
    /// The line did not have exactly three fields.  
    FieldCount(usize),  
    /// The SKU field was empty.  
    EmptySku,  
    /// A numeric field was not a whole number.  
    BadNumber {  
        /// Field name.  
        field: &'static str,  
        /// Text that was found.  
        value: String,  
    },  
}
```

```
impl fmt::Display for ParseError {
```

```
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {  
        match self {  
            ParseError::FieldCount(n) => {  
                write!(f, "expected 3 fields, found {n}")  
            }  
            ParseError::EmptySku => write!(f, "SKU is empty"),  
            ParseError::BadNumber { field, value } => {  
                write!(f, "{field} is not a whole number: {value:?}")  
            }  
        }  
    }  
}
```

```
impl std::error::Error for ParseError {}
```

```
fn number<T: std::str::FromStr>(<
```

```
    field: &'static str,  
    value: &str,  
) -> Result<T, ParseError> {  
    value.parse().map_err(|_| ParseError::BadNumber {  
        field,  
        value: value.to_string(),  
    })  
}
```

```
/// Parses `sku,qty,unit_cents`; surrounding spaces are ignored.
```

```
pub fn parse_record(line: &str) -> Result<Record, ParseError> {  
    let fields: Vec<&str> = line.split(',').map(str::trim).collect();  
    if fields.len() != 3 {  
        return Err(ParseError::FieldCount(fields.len()));  
    }  
    if fields[0].is_empty() {
```

```

        return Err(ParseError::EmptySku);
    }
    Ok(Record {
        sku: fields[0].to_string(),
        qty: number("qty", fields[1])?,
        unit_cents: number("unit_cents", fields[2])?,
    })
}

/// Parses every line, keeping valid records and numbered errors.
pub fn parse_all(text: &str) -> (Vec<Record>, Vec<(usize, ParseError)>) {
    let mut records = Vec::new();
    let mut errors = Vec::new();
    for (index, line) in text.lines().enumerate() {
        let line = line.trim();
        if line.is_empty() || line.starts_with("sku,") {
            continue;
        }
        match parse_record(line) {
            Ok(record) => records.push(record),
            Err(error) => errors.push((index + 1, error)),
        }
    }
    (records, errors)
}

```

Acceptance tests: Activity 10

tests/acceptance.rs (identical in starter/ and solution/)

```

use activity10::{parse_all, parse_record, ParseError, Record};

const SAMPLE: &str = include_str!("../data/stock.csv");

#[test]
fn parses_a_valid_line_with_spaces() {
    assert_eq!(
        parse_record(" ABC-0001 , 40 , 899 "),
        Ok(Record {
            sku: "ABC-0001".into(),
            qty: 40,
            unit_cents: 899
        })
    );
}

#[test]
fn reports_each_error_kind() {
    assert_eq!(parse_record("ABC-0001,4"), Err(ParseError::FieldCount(2)));
    assert_eq!(parse_record(",5,100"), Err(ParseError::EmptySku));
    assert_eq!(
        parse_record("ABC-0002,twelve,2499"),
        Err(ParseError::BadNumber {
            field: "qty",
            value: "twelve".into()
        })
    );
    assert!(matches!(
        parse_record("ABC-0002,-1,2499"),
        Err(ParseError::BadNumber { field: "qty", .. })
    ));
}

#[test]
fn error_messages_are_readable() {

```

```

assert_eq!(
    ParseError::FieldCount(2).to_string(),
    "expected 3 fields, found 2"
);
let bad = ParseError::BadNumber {
    field: "qty",
    value: "twelve".into(),
};
assert_eq!(bad.to_string(), "qty is not a whole number: \"twelve\"");
}

#[test]
fn parse_all_keeps_good_lines_and_numbers_bad_ones() {
    let (records, errors) = parse_all(SAMPLE);
    let skus: Vec<&str> = records.iter().map(|r| r.sku.as_str()).collect();
    assert_eq!(skus, ["ABC-0001", "ABC-0003"]);
    let lines: Vec<usize> = errors.iter().map(|(line, _)| *line).collect();
    assert_eq!(lines, [3, 4, 6]);
}

#[test]
fn question_mark_works_with_boxed_errors(
) -> Result<(), Box<dyn std::error::Error>> {
    let record = parse_record("ABC-0005,7,350")?;
    assert_eq!(record.qty, 7);
    Ok(())
}

```

Data file: data/stock.csv

```

sku,qty,unit_cents
ABC-0001,40,899
ABC-0002,twelve,2499
,5,100
ABC-0003,0,1299
ABC-0004,7

```

Expected result: cargo test reports 5 passed; data/stock.csv yields 2 records and errors on lines 3, 4 and 6.

Verified before release: solution 5 tests passed; starter 1 passed / 4 failed as intended.

Stretch task: Add ParseError::NegativeValue for "-1" so the message is clearer than BadNumber.

Deliverables: your completed starter crate passing every gate, one extra boundary test, docs/review-record.md and docs/verification-record.md.

Activity 11: Debug a failing integration with tests (Slides 164-168)

Folder: labs/activity-11-debug-integration. Maps to K4/A4. Suggested time: 35 minutes.

Goal and scenario: The nightly stock summary is wrong: the total is too low and a correctly stocked item is flagged. The starter crate contains two real defects. You reproduce them with tests, diagnose them with the help of an AI assistant, fix them and keep regression tests.

Exact contract: summarise(csv, threshold) -> Result<Summary, String> where Summary { lines, total_cents, low_stock }. Every line counts towards the total; low stock is strictly below the threshold, sorted by SKU; malformed lines are Err("line N: ...").

Step-by-step: Activity 11

1. Read the specification. Read the Goal and Exact contract sections below and the files in data/ (if any). Write down each input, its type and valid range, the failure behaviour and the boundary values before you open any code.

2. Open the activity in VS Code. Open this activity folder in Visual Studio Code with the rust-analyzer extension enabled. Confirm the toolchain with `rustc --version` and `cargo --version` in the integrated terminal.
3. Run the reference solution. In the terminal run: `cd solution && cargo test`. Confirm the result line shows every test passing. This proves your toolchain works and shows the target behaviour. (Starter: 4 of 6 tests fail. After both fixes cargo test reports 6 passed.)
4. Run the starter and read the failures. Run: `cd ../starter && cargo test`. Four of the six tests fail because the starter contains two real defects; the other two pass. Read the first failure: note the test name, the expected value and the actual value.
5. Plan with the AI assistant. Paste the "Plan before code" prompt from PROMPTS.md into your AI coding assistant (for example GitHub Copilot Chat, Claude or ChatGPT). Compare its edge cases with your own list from step 1 and record agreements and disagreements in docs/review-record.md.
6. Vibe-code the implementation. Use the "Generate with AI" prompt to generate code for starter/src/lib.rs. Review the proposal before accepting it: public signatures unchanged, no unsafe, no unwrap() or expect() on input data, no new crates, and every line explained. Reject or edit anything you cannot justify.
7. Test until green. Run cargo test after each accepted change. If a test fails or the code does not compile, use the "Repair from evidence" prompt with the exact cargo output. Never edit expected values in tests to make them pass.
8. Apply the quality gates. Run every gate from the starter folder: `cargo test ; cargo clippy --all-targets -- -D warnings ; cargo fmt --check`. Fix each clippy warning and formatting difference; these are the organisation's coding standards.
9. Add your own boundary test. Predict one more boundary case from the specification and add it to the test file in starter/tests/. Run it against both starter and solution. Stretch: Add a test for threshold 0 (nothing is low) and explain why it passed even before the fix.
10. Record the evidence. Complete docs/verification-record.md with the commands you ran, the exact result lines (for example "test result: ok. 5 passed"), the AI proposals you accepted or rejected and why. Compare your code with solution/ only after your own tests pass.

Commands: Activity 11

```
cd solution && cargo test
cd ../starter
cargo test
cargo clippy --all-targets -- -D warnings
cargo fmt --check
```

Vibe-coding prompts: Activity 11

Plan before code: Act as a senior Rust reviewer. Specification: `summarise(csv, threshold) -> Result<Summary, String>` where `Summary { lines, total_cents, low_stock }`. Every line counts towards the total; low stock is strictly below the threshold, sorted by SKU; malformed lines are `Err("line N: ...")`. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI: Implement "Debug a failing integration with tests" in starter/src so that the supplied tests pass. Give the AI the failing test output and the function only; ask for a diagnosis before any code change. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence: Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

The same prompts are in labs/activity-11-debug-integration/PROMPTS.md and Vibe-Coding-Prompts.pdf.

Reference solution: Activity 11

solution/src/lib.rs

```
///! Activity 11: an integrated stock summary (the starter has two seeded defects).

/// One parsed stock line.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Line {
    /// Product code.
    pub sku: String,
    /// Units on hand.
    pub qty: u32,
    /// Unit cost in cents.
    pub unit_cents: u64,
}

/// Result of the nightly summary.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Summary {
    /// Number of stock lines.
    pub lines: usize,
    /// Total stock value in cents.
    pub total_cents: u64,
    /// SKUs strictly below the threshold, sorted.
    pub low_stock: Vec<String>,
}

/// Parses `sku,qty,unit_cents` lines, skipping the header and blanks.
pub fn parse_lines(csv: &str) -> Result<Vec<Line>, String> {
    let mut lines = Vec::new();
    for (index, raw) in csv.lines().enumerate() {
        let raw = raw.trim();
        if raw.is_empty() || raw.starts_with("sku,") {
            continue;
        }
        let fields: Vec<&str> = raw.split(',').map(str::trim).collect();
        let [sku, qty, unit] = fields.as_slice() else {
            return Err(format!("line {}: expected 3 fields", index + 1));
        };
        let qty = qty
            .parse()
            .map_err(|_| format!("line {}: bad qty {qty:?}", index + 1))?;
        let unit_cents = unit
            .parse()
            .map_err(|_| format!("line {}: bad unit {unit:?}", index + 1))?;
        lines.push(Line {
            sku: sku.to_string(),
            qty,
            unit_cents,
        });
    }
    Ok(lines)
}

/// Total value of every line, or `None` on overflow.
pub fn total_cents(lines: &[Line]) -> Option<u64> {
```

```

        lines.iter().try_fold(0u64, |total, line| {
            total.checked_add(line.unit_cents.checked_mul(u64::from(line.qty)))?
        })
    }

    /// SKUs strictly below `threshold`, sorted.
    pub fn low_stock(lines: &[Line], threshold: u32) -> Vec<String> {
        let mut skus: Vec<String> = lines
            .iter()
            .filter(|line| line.qty < threshold)
            .map(|line| line.sku.clone())
            .collect();
        skus.sort();
        skus
    }

    /// Parses, totals and flags low stock in one call.
    pub fn summarise(csv: &str, threshold: u32) -> Result<Summary, String> {
        let lines = parse_lines(csv)?;
        let total_cents =
            total_cents(&lines).ok_or("stock value overflowed u64")?;
        Ok(Summary {
            lines: lines.len(),
            total_cents,
            low_stock: low_stock(&lines, threshold),
        })
    }
}

```

Acceptance tests: Activity 11

tests/integration.rs (identical in starter/ and solution/)

```

use activity11::{low_stock, parse_lines, summarise, total_cents, Summary};

const SAMPLE: &str = include_str!("../data/stock.csv");

#[test]
fn parses_every_stock_line() {
    assert_eq!(parse_lines(SAMPLE).unwrap().len(), 4);
}

#[test]
fn total_includes_the_first_line() {
    let lines = parse_lines(SAMPLE).unwrap();
    assert_eq!(total_cents(&lines), Some(52_655));
}

#[test]
fn low_stock_is_strictly_below_threshold() {
    let lines = parse_lines(SAMPLE).unwrap();
    assert_eq!(low_stock(&lines, 5), ["ABC-0003"]);
}

#[test]
fn end_to_end_summary() {
    assert_eq!(
        summarise(SAMPLE, 5),
        Ok(Summary {
            lines: 4,
            total_cents: 52_655,
            low_stock: vec!["ABC-0003".to_string()],
        })
    );
}

```

```
#[test]
fn malformed_line_reports_its_number() {
    assert_eq!(
        parse_lines("sku,qty,unit_cents\nABC-0001,4"),
        Err("line 2: expected 3 fields".to_string())
    );
}

#[test]
fn overflow_is_an_error_not_a_panic() {
    assert_eq!(
        summarise("ABC-0001,2,18446744073709551615", 0),
        Err("stock value overflowed u64".to_string())
    );
}
```

Data file: data/stock.csv

```
sku,qty,unit_cents
ABC-0001,40,899
ABC-0002,5,2499
ABC-0003,0,1299
ABC-0004,12,350
```

Expected result: Starter: 4 of 6 tests fail. After both fixes cargo test reports 6 passed.

Verified before release: solution 6 tests passed; starter 2 passed / 4 failed as intended.

Stretch task: Add a test for threshold 0 (nothing is low) and explain why it passed even before the fix.

Deliverables: your completed starter crate passing every gate, one extra boundary test, docs/review-record.md and docs/verification-record.md.

Activity 12: Integrate components into a tested command-line tool (Slides 169-173)

Folder: labs/activity-12-cli-integration. Maps to K4/A4. Suggested time: 30 minutes.

Goal and scenario: Operations staff will run StockPilot from a terminal and from scheduled scripts. You connect the library to a binary that reads a file, prints a report and returns exit codes that other programs can rely on.

Exact contract: `run(csv, threshold) -> Result<String, String>` returns "ITEMS n\nTOTAL \$D.CC\nLOW STOCK a, b | none". The binary takes `<file> [threshold=5]`; exit 0 on success, 1 for unreadable or invalid data, 2 for usage errors.

Step-by-step: Activity 12

1. Read the specification. Read the Goal and Exact contract sections below and the files in data/ (if any). Write down each input, its type and valid range, the failure behaviour and the boundary values before you open any code.
2. Open the activity in VS Code. Open this activity folder in Visual Studio Code with the rust-analyzer extension enabled. Confirm the toolchain with `rustc --version` and `cargo --version` in the integrated terminal.
3. Run the reference solution. In the terminal run: `cd solution && cargo test`. Confirm the result line shows every test passing. This proves your toolchain works and shows the target behaviour. (cargo test reports 7 passed; cargo run -- ../data/stock.csv prints ITEMS 4, TOTAL \$526.55, LOW STOCK ABC-0003.)
4. Run the starter and read the failures. Run: `cd ../starter && cargo test`. Every test that calls an unfinished function fails with "not yet implemented" from `todo!()`. This is expected: the tests are the specification you will implement against.

5. Plan with the AI assistant. Paste the "Plan before code" prompt from PROMPTS.md into your AI coding assistant (for example GitHub Copilot Chat, Claude or ChatGPT). Compare its edge cases with your own list from step 1 and record agreements and disagreements in docs/review-record.md.
6. Vibe-code the implementation. Use the "Generate with AI" prompt to generate code for starter/src/lib.rs. Review the proposal before accepting it: public signatures unchanged, no unsafe, no unwrap() or expect() on input data, no new crates, and every line explained. Reject or edit anything you cannot justify.
7. Test until green. Run cargo test after each accepted change. If a test fails or the code does not compile, use the "Repair from evidence" prompt with the exact cargo output. Never edit expected values in tests to make them pass.
8. Apply the quality gates. Run every gate from the starter folder: cargo test ; cargo run -- ../data/stock.csv ; cargo clippy --all-targets -- -D warnings ; cargo fmt --check. Fix each clippy warning and formatting difference; these are the organisation's coding standards.
9. Add your own boundary test. Predict one more boundary case from the specification and add it to the test file in starter/tests/. Run it against both starter and solution. Stretch: Add a --json flag that prints the same report as JSON, with one new process test.
10. Record the evidence. Complete docs/verification-record.md with the commands you ran, the exact result lines (for example "test result: ok. 5 passed"), the AI proposals you accepted or rejected and why. Compare your code with solution/ only after your own tests pass.

Commands: Activity 12

```
cd solution && cargo test
cd ../starter
cargo test
cargo run -- ../data/stock.csv
cargo clippy --all-targets -- -D warnings
cargo fmt --check
```

Vibe-coding prompts: Activity 12

Plan before code: Act as a senior Rust reviewer. Specification: run(csv, threshold) -> Result<String, String> returns "ITEMS n\nTOTAL \$D.CC\nLOW STOCK a, b | none". The binary takes <file> [threshold=5]; exit 0 on success, 1 for unreadable or invalid data, 2 for usage errors. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI: Implement "Integrate components into a tested command-line tool" in starter/src so that the supplied tests pass. Keep main thin; put every rule in lib.rs where the library tests can reach it. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence: Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

The same prompts are in labs/activity-12-cli-integration/PROMPTS.md and Vibe-Coding-Prompts.pdf.

Reference solution: Activity 12

solution/src/lib.rs

```
//! Activity 12: library logic behind a command-line report.

fn money(cents: u64) -> String {
    format!("{${}.{:02}", cents / 100, cents % 100)
```

```

}

/// Builds the three-line stock report from CSV text.
pub fn run(csv: &str, threshold: u32) -> Result<String, String> {
    let mut count = 0usize;
    let mut total = 0u64;
    let mut low = Vec::new();
    for (index, raw) in csv.lines().enumerate() {
        let raw = raw.trim();
        if raw.is_empty() || raw.starts_with("sku,") {
            continue;
        }
        let fields: Vec<&str> = raw.split(',').map(str::trim).collect();
        let [sku, qty, unit] = fields.as_slice() else {
            return Err(format!("line {}: expected 3 fields", index + 1));
        };
        let qty: u32 = qty
            .parse()
            .map_err(|_| format!("line {}: bad qty", index + 1))?;
        let unit: u64 = unit
            .parse()
            .map_err(|_| format!("line {}: bad unit", index + 1))?;
        total = unit
            .checked_mul(u64::from(qty))
            .and_then(|value| total.checked_add(value))
            .ok_or("stock value overflowed u64")?;
        count += 1;
        if qty < threshold {
            low.push(sku.to_string());
        }
    }
    low.sort();
    let low = if low.is_empty() {
        "none".to_string()
    } else {
        low.join(", ")
    };
    Ok(format!(
        "ITEMS {count}\nTOTAL {}\nLOW STOCK {low}",
        money(total)
    ))
}

```

solution/src/main.rs

```

use std::process::ExitCode;

fn main() -> ExitCode {
    let args: Vec<String> = std::env::args().skip(1).collect();
    let (path, threshold) = match args.as_slice() {
        [path] => (path.as_str(), 5),
        [path, threshold] => match threshold.parse() {
            Ok(value) => (path.as_str(), value),
            Err(_) => {
                eprintln!("error: threshold must be a whole number");
                return ExitCode::from(2);
            }
        },
        _ => {
            eprintln!("usage: activity12 <stock.csv> [low-stock-threshold]");
            return ExitCode::from(2);
        }
    };
    let text = match std::fs::read_to_string(path) {
        Ok(text) => text,
        Err(error) => {

```

```

        eprintln!("error: cannot read {path}: {error}");
        return ExitCode::from(1);
    }
};
match activity12::run(&text, threshold) {
    Ok(report) => {
        println!("{report}");
        ExitCode::SUCCESS
    }
    Err(message) => {
        eprintln!("error: {message}");
        ExitCode::from(1)
    }
}
}
}

```

Acceptance tests: Activity 12

tests/cli.rs (identical in starter/ and solution/)

```

use std::process::Command;

const DATA: &str = concat!(env!("CARGO_MANIFEST_DIR"), "../data/stock.csv");
const REPORT: &str = "ITEMS 4\nTOTAL $526.55\nLOW STOCK ABC-0003";

fn stockpilot() -> Command {
    Command::new(env!("CARGO_BIN_EXE_activity12"))
}

#[test]
fn library_report_matches_the_specification() {
    assert_eq!(
        activity12::run(include_str!("../data/stock.csv"), 5),
        Ok(REPORT.to_string())
    );
}

#[test]
fn empty_input_reports_zero() {
    assert_eq!(
        activity12::run("", 5),
        Ok("ITEMS 0\nTOTAL $0.00\nLOW STOCK none".to_string())
    );
}

#[test]
fn malformed_row_is_an_error() {
    assert_eq!(
        activity12::run("ABC-0001,1", 5),
        Err("line 1: expected 3 fields".to_string())
    );
}

#[test]
fn binary_prints_the_report_and_exits_zero() {
    let output = stockpilot().arg(DATA).output().expect("binary runs");
    assert!(output.status.success());
    assert_eq!(String::from_utf8_lossy(&output.stdout).trim_end(), REPORT);
}

#[test]
fn binary_accepts_a_threshold() {
    let output = stockpilot()
        .args([DATA, "13"])

```

```

        .output()
        .expect("binary runs");
    let stdout = String::from_utf8_lossy(&output.stdout);
    assert!(stdout.contains("LOW STOCK ABC-0002, ABC-0003, ABC-0004"));
}

#[test]
fn usage_error_exits_two() {
    let output = stockpilot().output().expect("binary runs");
    assert_eq!(output.status.code(), Some(2));
    assert!(String::from_utf8_lossy(&output.stderr).contains("usage"));
}

#[test]
fn unreadable_file_exits_one() {
    let output = stockpilot()
        .arg("no-such-file.csv")
        .output()
        .expect("binary runs");
    assert_eq!(output.status.code(), Some(1));
}

```

Data file: data/stock.csv

```

sku,qty,unit_cents
ABC-0001,40,899
ABC-0002,5,2499
ABC-0003,0,1299
ABC-0004,12,350

```

Expected result: cargo test reports 7 passed; cargo run -- ../data/stock.csv prints ITEMS 4, TOTAL \$526.55, LOW STOCK ABC-0003.

Verified before release: solution 7 tests passed; starter 2 passed / 5 failed as intended.

Stretch task: Add a --json flag that prints the same report as JSON, with one new process test.

Deliverables: your completed starter crate passing every gate, one extra boundary test, docs/review-record.md and docs/verification-record.md.

Topic 5: AI-Assisted Rust Software Design and Documentation (Slides 175-207)

K5/A5 / LO5: Generate programming design documentation aligned with user specifications (K5/A5).

Doc comments become documentation (Slides 176-178)

///
//!: Documents the enclosing item: the crate or module.

///
///: Documents the next item: function, struct, field.

Markdown: Headings, lists, code and links render in cargo doc.

deny(missing_docs): An undocumented public item fails the build.

```

///  
//! Reorder-point calculations for StockPilot.    (crate docs)  
#[deny(missing_docs)]  
  
///  
/// Calculates the reorder point in units.  
///  
/// `point = daily_usage x lead_days + safety_stock` (R1).  
/// Returns `None` instead of wrapping on overflow (R2).  
pub fn reorder_point(daily_usage: u32, lead_days: u32,  
                    safety_stock: u32) -> Option<u32>

```

```
cargo doc --no-deps --open
```

Contract field	Exact rule
Summary line	One sentence saying what it does.
Units and ranges	State units (cents, days) and limits.
Failure	Say what None/Err means, or add # Panics / # Errors.
Trace	Reference requirement IDs such as R1.

Common failure: warning: missing documentation for a function. What to do: Add a /// summary with units, return meaning and an example.

How you know it works: cargo doc builds and clippy -D warnings reports no missing docs.

Doctests keep examples honest (Slides 179-181)

Doctest: A code block in docs is compiled and run by cargo test.

Honesty: If behaviour changes, the stale example fails the build.

Users first: Examples show how to call the API correctly.

AI drafts: AI-written examples must pass as doctests before acceptance.

```
/// # Examples
///
/// ```
/// use activity13::reorder_point;
/// assert_eq!(reorder_point(12, 5, 20), Some(80));
/// ```
pub fn reorder_point(..) -> Option<u32> { .. }
```

```
$ cargo test
    Doc-tests activity13
test src/lib.rs - reorder_point (line 12) ... ok
```

Contract field	Exact rule
Location	Under a # Examples heading in /// comments.
Imports	Doctests use the crate like an external user.
Hidden lines	Lines starting with # are compiled but not shown.
Evidence	The Doc-tests section of cargo test output.

Common failure: Doctest fails after a code change. What to do: Update the example to the new, specified behaviour, or fix the regression it caught.

How you know it works: Activity 13: 3 unit tests and 3 doctests pass.

Trace requirements to code and tests (Slides 182-184)

Trace matrix: Each user requirement links to code and a named test.

Coverage gap: An empty cell is a missing test or feature.

Change impact: Changing R3 shows exactly which code and tests to update.

Audit: Reviewers can verify claims without reading all code.

ID	Requirement	Test (tests/acceptance.rs)
R1	point = usage*lead+safe	r1_r3_below_the_point...
R2	above point: no order	r2_above_the_point...
R3	order up to 7-day target	r2_exactly_at_the_point...
R4	overflow item skipped	r4_overflow_is_skipped...

```
Code: reorder_suggestions() in src/lib.rs
R1 checked_mul(..)?.checked_add(..)?
R2 if item.on_hand > point { return None; }
R3 point + cover - on_hand, kept only if > 0
R4 ? after every checked_* call
```

Contract field	Exact rule
Requirement ID	Stable identifiers R1..Rn used in docs, tests and commits.
Code	Function or line that implements it.
Test	Exact test name; the result must be ok.
Status	Passed, failed or not yet run: never assumed.

Common failure: Requirement without a test. What to do: Add a test named after the requirement before claiming it is met.

How you know it works: Activity 14 design.md traces R1-R4 to named, passing tests.

Architecture and module maps (Slides 185-187)

Components: One box per module with a single responsibility.

Data flow: Arrows labelled with the Rust types that cross them.

Boundaries: I/O at the edges; pure logic in the middle.

Mermaid: Diagrams as text in Markdown, versioned with the code.

```
+-----+
CSV/JSON ->| parse      |-- Result<Vec<Line>, Error>
+-----+
      |
+-----+ +-----+
| inventory |---->| report      |--> stdout
+-----+ +-----+
      |
+-----+
| reorder   |--> Vec<Suggestion>
+-----+
```

Contract field	Exact rule
Module	Name, responsibility, public functions.
Interface	Input type, output type, error type.
External	Files, arguments, exit codes, crates.
Owner	Who maintains it and where its tests live.

Common failure: Diagram disagrees with the code. What to do: Generate the module list from lib.rs and update the diagram in the same change.

How you know it works: Every box in design.md exists as a module or function in the crate.

Record design decisions (Slides 188-190)

Decision: What was decided, in one sentence.

Options: Alternatives considered, so reviewers see the trade-off.

Consequences: Benefits and costs, including follow-up work.

Evidence: The test that demonstrates the decision.

Decision: Skip items whose arithmetic overflows (R4).

Context: Planning data can contain bad usage values.
Options: 1. panic 2. saturate at u32::MAX 3. skip item
Chosen: 3 - skip; wrong orders are worse than none.
Consequences:
+ no crash, no absurd order quantities
- skipped items must be visible in a report (future)
Evidence: test r4_overflow_is_skipped_without_panic

Contract field	Exact rule
When	Any choice a maintainer would question later.
Where	docs/design.md next to the code.
AI role	AI may list options; you choose and justify.
Update	Supersede, do not delete, old decisions.

Common failure: Nobody knows why the code behaves this way. What to do: Add a decision record with context, options and a linked test.

How you know it works: design.md contains at least one complete decision record.

AI-assisted documentation, human-verified (Slides 191-193)

Grounding: Give the AI the code and requirements, nothing else.

Hallucination: AI may describe behaviour the code does not have.

Doctests: Executable examples catch false claims automatically.

Ownership: You sign off the documentation, not the AI.

Prompt: "Draft rustdoc for reorder_suggestions using ONLY this code and R1-R4. Include units, a # Examples doctest and a requirement list. Mark anything you are unsure of."

Verify:
cargo test # doctest compiles and passes
cargo doc --no-deps # renders
compare every claim with code and tests

Contract field	Exact rule
Accept	Claims that match code and a passing test.
Correct	Wrong units, ranges or missing failure cases.
Reject	Invented functions, options or performance claims.
Record	Section, what was verified, what was changed.

Common failure: Docs claim a feature that does not exist. What to do: Delete the claim or implement and test it; never leave it unverified.

How you know it works: The AI review table in design.md lists verified and corrected sections.

Hand over: README and maintenance notes (Slides 194-196)

Audience: Written for the maintainer who joins next month.

Commands: Exact commands that prove the system still works.

Limits: Honest list of what is not handled yet.

Procedure: How to make a change safely.

README.md
What it does (one paragraph)
Build and test cargo test / clippy / fmt

Usage	stockpilot <file> [threshold]
Requirements trace	link to docs/design.md
Known limits	e.g. skipped overflow items
Change procedure	update code + docs + tests together

Contract field	Exact rule
Build	Toolchain version and commands.
Interfaces	Inputs, outputs, exit codes and data formats.
Quality gates	fmt, clippy -D warnings, test, doc.
Contacts	Owner and where decisions are recorded.

Common failure: Handover depends on the original developer. What to do: Write down commands, limits and decisions until a newcomer can run and change it alone.

How you know it works: A classmate can build, test and explain the capstone from its README and design.md.

Activity 13: Document an API with rustdoc and doctests (Slides 197-201)

Folder: labs/activity-13-rustdoc-doctests. Maps to K5/A5. Suggested time: 30 minutes.

Goal and scenario: The maintenance team will own the reorder calculation. The starter code works but has no documentation. You write rustdoc comments whose examples are compiled and run as tests, and trace them to requirements.

Exact contract: `reorder_point(daily_usage, lead_days, safety_stock) -> Option<u32> = daily_usage x lead_days + safety_stock (R1), None on overflow (R2). needs_reorder(on_hand, point) is true at or below the point (R3). Every public item is documented; cargo clippy with -D warnings enforces missing_docs.`

Step-by-step: Activity 13

1. Read the specification. Read the Goal and Exact contract sections below and the files in data/ (if any). Write down each input, its type and valid range, the failure behaviour and the boundary values before you open any code.
2. Open the activity in VS Code. Open this activity folder in Visual Studio Code with the rust-analyzer extension enabled. Confirm the toolchain with `rustc --version` and `cargo --version` in the integrated terminal.
3. Run the reference solution. In the terminal run: `cd solution && cargo test`. Confirm the result line shows every test passing. This proves your toolchain works and shows the target behaviour. (cargo test runs 3 tests and 3 doctests; cargo doc --no-deps builds; clippy -D warnings is clean.)
4. Run the starter and read the failures. Run: `cd ../starter && cargo test`. cargo test passes because the logic already works, but cargo clippy --all-targets -- -D warnings fails with "missing documentation" errors. Your job is the documentation.
5. Plan with the AI assistant. Paste the "Plan before code" prompt from PROMPTS.md into your AI coding assistant (for example GitHub Copilot Chat, Claude or ChatGPT). Compare its edge cases with your own list from step 1 and record agreements and disagreements in docs/review-record.md.
6. Vibe-code the implementation. Use the "Generate with AI" prompt to generate code for starter/src/lib.rs. Review the proposal before accepting it: public signatures unchanged, no unsafe, no unwrap() or expect() on input data, no new crates, and every line explained. Reject or edit anything you cannot justify.

7. Test until green. Run cargo test after each accepted change. If a test fails or the code does not compile, use the "Repair from evidence" prompt with the exact cargo output. Never edit expected values in tests to make them pass.
8. Apply the quality gates. Run every gate from the starter folder: cargo test ; cargo doc --no-deps ; cargo clippy --all-targets -- -D warnings ; cargo fmt --check. Fix each clippy warning and formatting difference; these are the organisation's coding standards.
9. Add your own boundary test. Predict one more boundary case from the specification and add it to the test file in starter/tests/. Run it against both starter and solution. Stretch: Add a # Panics or # Errors section policy to docs/design-note.md and justify why neither applies here.
10. Record the evidence. Complete docs/verification-record.md with the commands you ran, the exact result lines (for example "test result: ok. 5 passed"), the AI proposals you accepted or rejected and why. Compare your code with solution/ only after your own tests pass.

Commands: Activity 13

```
cd solution && cargo test
cd ../starter
cargo test
cargo doc --no-deps
cargo clippy --all-targets -- -D warnings
cargo fmt --check
```

Vibe-coding prompts: Activity 13

Plan before code: Act as a senior Rust reviewer. Specification: reorder_point(daily_usage, lead_days, safety_stock) -> Option<u32> = daily_usage x lead_days + safety_stock (R1), None on overflow (R2). needs_reorder(on_hand, point) is true at or below the point (R3). Every public item is documented; cargo clippy with -D warnings enforces missing_docs. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI: Implement "Document an API with rustdoc and doctests" in starter/src so that the supplied tests pass. Ask the AI to draft docs from the code and requirements only; verify every example by running cargo test. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence: Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

The same prompts are in labs/activity-13-rustdoc-doctests/PROMPTS.md and Vibe-Coding-Prompts.pdf.

Reference solution: Activity 13

solution/src/lib.rs

```
//! Reorder-point calculations for the StockPilot stock service.
//!
//! Requirement trace: R1 reorder-point formula, R2 overflow is reported
//! instead of wrapping, R3 stock at or below the point needs reordering.
#![deny(missing_docs)]

/// Calculates the reorder point in units.
///
/// `reorder point = daily_usage x lead_days + safety_stock` (R1).
///
/// # Examples
///
```

```

/// ```
/// use activity13::reorder_point;
/// assert_eq!(reorder_point(12, 5, 20), Some(80));
/// ```
///
/// Returns `None` instead of wrapping when the result exceeds `u32` (R2):
///
/// ```
/// use activity13::reorder_point;
/// assert_eq!(reorder_point(u32::MAX, 2, 0), None);
/// ```
pub fn reorder_point(
    daily_usage: u32,
    lead_days: u32,
    safety_stock: u32,
) -> Option<u32> {
    daily_usage
        .checked_mul(lead_days)?
        .checked_add(safety_stock)
}

/// Reports whether stock on hand has reached the reorder point (R3).
///
/// Stock exactly at the reorder point needs reordering.
///
/// # Examples
///
/// ```
/// use activity13::needs_reorder;
/// assert!(needs_reorder(80, 80));
/// assert!(needs_reorder(81, 80));
/// ```
pub fn needs_reorder(on_hand: u32, reorder_point: u32) -> bool {
    on_hand <= reorder_point
}

```

Acceptance tests: Activity 13

tests/acceptance.rs (identical in starter/ and solution/)

```

use activity13::{needs_reorder, reorder_point};

#[test]
fn r1_formula() {
    assert_eq!(reorder_point(12, 5, 20), Some(80));
    assert_eq!(reorder_point(0, 5, 20), Some(20));
}

#[test]
fn r2_overflow_is_reported() {
    assert_eq!(reorder_point(u32::MAX, 2, 0), None);
    assert_eq!(reorder_point(1, u32::MAX, 1), None);
}

#[test]
fn r3_boundary() {
    assert!(needs_reorder(79, 80));
    assert!(needs_reorder(80, 80));
    assert!(needs_reorder(81, 80));
}

```

Expected result: cargo test runs 3 tests and 3 doctests; cargo doc --no-deps builds; clippy -D warnings is clean.

Verified before release: solution 6 tests passed (including 3 doctests); starter 3 passed / 0 failed as intended.

Stretch task: Add a # Panics or # Errors section policy to docs/design-note.md and justify why neither applies here.

Deliverables: your completed starter crate passing every gate, one extra boundary test, docs/review-record.md and docs/verification-record.md.

Activity 14: Design and document a reorder feature with AI (Slides 202-206)

Folder: labs/activity-14-design-capstone. Maps to K5/A5 K4/A4. Suggested time: 30 minutes.

Goal and scenario: Capstone: purchasing wants weekly reorder suggestions. You agree requirements R1-R4, implement the feature with an AI assistant, and hand over a design record that traces every requirement to code and tests.

Exact contract: `reorder_suggestions(&[StockItem]) -> Vec<Suggestion>`. R1 point = usage x lead + safety; R2 items above the point are excluded; R3 `order_qty` = point + 7 days usage - `on_hand`, zero quantities excluded; R4 overflowing items are skipped, never wrapped. Output sorted by SKU.

Step-by-step: Activity 14

1. Read the specification. Read the Goal and Exact contract sections below and the files in data/ (if any). Write down each input, its type and valid range, the failure behaviour and the boundary values before you open any code.
2. Open the activity in VS Code. Open this activity folder in Visual Studio Code with the rust-analyzer extension enabled. Confirm the toolchain with `rustc --version` and `cargo --version` in the integrated terminal.
3. Run the reference solution. In the terminal run: `cd solution && cargo test`. Confirm the result line shows every test passing. This proves your toolchain works and shows the target behaviour. (cargo test reports 6 passed and 1 doctest; docs/design.md traces R1-R4 to named tests.)
4. Run the starter and read the failures. Run: `cd ../starter && cargo test`. Every test that calls an unfinished function fails with "not yet implemented" from `todo!()`. This is expected: the tests are the specification you will implement against.
5. Plan with the AI assistant. Paste the "Plan before code" prompt from PROMPTS.md into your AI coding assistant (for example GitHub Copilot Chat, Claude or ChatGPT). Compare its edge cases with your own list from step 1 and record agreements and disagreements in docs/review-record.md.
6. Vibe-code the implementation. Use the "Generate with AI" prompt to generate code for `starter/src/lib.rs`. Review the proposal before accepting it: public signatures unchanged, no `unsafe`, no `unwrap()` or `expect()` on input data, no new crates, and every line explained. Reject or edit anything you cannot justify.
7. Test until green. Run cargo test after each accepted change. If a test fails or the code does not compile, use the "Repair from evidence" prompt with the exact cargo output. Never edit expected values in tests to make them pass.
8. Apply the quality gates. Run every gate from the starter folder: `cargo test` ; `cargo doc --no-deps` ; `cargo clippy --all-targets -- -D warnings` ; `cargo fmt --check`. Fix each clippy warning and formatting difference; these are the organisation's coding standards.
9. Add your own boundary test. Predict one more boundary case from the specification and add it to the test file in `starter/tests/`. Run it against both starter and solution. Stretch: Add R5: cap any single order at 1,000 units; update the code, docs, trace table and tests together.

10. Record the evidence. Complete docs/verification-record.md with the commands you ran, the exact result lines (for example "test result: ok. 5 passed"), the AI proposals you accepted or rejected and why. Compare your code with solution/ only after your own tests pass.

Commands: Activity 14

```
cd solution && cargo test
cd ../starter
cargo test
cargo doc --no-deps
cargo clippy --all-targets -- -D warnings
cargo fmt --check
```

Vibe-coding prompts: Activity 14

Plan before code: Act as a senior Rust reviewer. Specification: `reorder_suggestions(&[StockItem]) -> Vec<Suggestion>`. R1 point = usage x lead + safety; R2 items above the point are excluded; R3 `order_qty` = point + 7 days usage - `on_hand`, zero quantities excluded; R4 overflowing items are skipped, never wrapped. Output sorted by SKU. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI: Implement "Design and document a reorder feature with AI" in starter/src so that the supplied tests pass. Give the AI R1-R4 and the public types; require a trace table in its answer and verify each row against the tests. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence: Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

The same prompts are in labs/activity-14-design-capstone/PROMPTS.md and Vibe-Coding-Prompts.pdf.

Reference solution: Activity 14

solution/src/lib.rs

```
///! Reorder suggestions designed and documented with AI assistance.
///!
///! See `docs/design.md` for the requirement trace (R1-R4).
#![deny(missing_docs)]

/// One stock-keeping unit and its planning inputs.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct StockItem {
    /// Product code, for example `ABC-0001`.
    pub sku: String,
    /// Units currently on hand.
    pub on_hand: u32,
    /// Average units used per day.
    pub daily_usage: u32,
    /// Supplier lead time in days.
    pub lead_days: u32,
    /// Buffer stock kept for demand spikes.
    pub safety_stock: u32,
}

/// A proposed purchase-order line.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct Suggestion {
    /// Product code.
    pub sku: String,
```

```

    /// Units to order.
    pub order_qty: u32,
}

/// Days of usage the order should cover beyond the reorder point (R3).
pub const REVIEW_DAYS: u32 = 7;

/// Suggests purchase quantities for items at or below their reorder point.
///
/// * R1: reorder point = `daily_usage x lead_days + safety_stock`.
/// * R2: items above their reorder point are not suggested.
/// * R3: order quantity = reorder point + [`REVIEW_DAYS`] of usage - on hand;
///       zero quantities are not suggested.
/// * R4: items whose arithmetic overflows are skipped, never wrapped.
///
/// Suggestions are sorted by SKU.
///
/// # Examples
///
/// ```
/// use activity14::{reorder_suggestions, StockItem};
/// let item = StockItem {
///     sku: "ABC-0001".into(),
///     on_hand: 50,
///     daily_usage: 10,
///     lead_days: 5,
///     safety_stock: 20,
/// };
/// assert_eq!(reorder_suggestions(&[item])[0].order_qty, 90);
/// ```
pub fn reorder_suggestions(items: &[StockItem]) -> Vec<Suggestion> {
    let mut suggestions: Vec<Suggestion> = items
        .iter()
        .filter_map(|item| {
            let point = item
                .daily_usage
                .checked_mul(item.lead_days)?
                .checked_add(item.safety_stock)?;
            if item.on_hand > point {
                return None;
            }
            let cover = item.daily_usage.checked_mul(REVIEW_DAYS)?;
            let order_qty = point.checked_add(cover)? - item.on_hand;
            (order_qty > 0).then(|| Suggestion {
                sku: item.sku.clone(),
                order_qty,
            })
        })
        .collect();
    suggestions.sort_by(|a, b| a.sku.cmp(&b.sku));
    suggestions
}

```

Acceptance tests: Activity 14

tests/acceptance.rs (identical in starter/ and solution/)

```

use activity14::{reorder_suggestions, StockItem, Suggestion};

fn item(
    sku: &str,
    on_hand: u32,
    daily_usage: u32,
    lead_days: u32,
    safety_stock: u32,
) -> StockItem {

```

```

    StockItem {
        sku: sku.into(),
        on_hand,
        daily_usage,
        lead_days,
        safety_stock,
    }
}

fn suggestion(sku: &str, order_qty: u32) -> Suggestion {
    Suggestion {
        sku: sku.into(),
        order_qty,
    }
}

#[test]
fn r1_r3_below_the_point_orders_up_to_target() {
    assert_eq!(
        reorder_suggestions(&[item("ABC-0001", 50, 10, 5, 20)]),
        [suggestion("ABC-0001", 90)]
    );
}

#[test]
fn r2_above_the_point_is_not_suggested() {
    assert!(reorder_suggestions(&[item("ABC-0002", 71, 10, 5, 20)]).is_empty());
}

#[test]
fn r2_exactly_at_the_point_is_suggested() {
    assert_eq!(
        reorder_suggestions(&[item("ABC-0003", 70, 10, 5, 20)]),
        [suggestion("ABC-0003", 70)]
    );
}

#[test]
fn r3_zero_quantity_is_not_suggested() {
    assert!(reorder_suggestions(&[item("ABC-0005", 0, 0, 10, 0)]).is_empty());
}

#[test]
fn r4_overflow_is_skipped_without_panic() {
    let items = [
        item("ABC-0006", 0, u32::MAX, 2, 0),
        item("ABC-0007", 0, 1, 1, 0),
    ];
    assert_eq!(reorder_suggestions(&items), [suggestion("ABC-0007", 8)]);
}

#[test]
fn output_is_sorted_by_sku() {
    let items = [item("ZZZ-0001", 0, 1, 1, 0), item("AAA-0001", 0, 1, 1, 0)];
    let skus: Vec<String> = reorder_suggestions(&items)
        .into_iter()
        .map(|s| s.sku)
        .collect();
    assert_eq!(skus, ["AAA-0001", "ZZZ-0001"]);
}

```

Expected result: cargo test reports 6 passed and 1 doctest; docs/design.md traces R1-R4 to named tests.

Verified before release: solution 7 tests passed (including 1 doctests); starter 0 passed / 7 failed as intended.

Stretch task: Add R5: cap any single order at 1,000 units; update the code, docs, trace table and tests together.

Deliverables: your completed starter crate passing every gate, one extra boundary test, docs/review-record.md and docs/verification-record.md.

Quick Command Reference

Command	Purpose
cargo new name	Create a new package
cargo build	Compile (debug)
cargo run -- args	Build and run the binary with arguments
cargo test	Run unit, integration and documentation tests
cargo test name -- --nocapture	Run matching tests and show their output
cargo clippy --all-targets -- -D warnings	Lint; warnings fail the check
cargo fmt / cargo fmt --check	Format / verify formatting
cargo doc --no-deps --open	Build and open API documentation
cargo add serde --features derive	Add a dependency
rustc --explain E0382	Explain a compiler error code
RUST_BACKTRACE=1 cargo test	Show the call path of a panic

AI Review Checklist

- Public signatures and the specification are unchanged.
- No unsafe; no unwrap() or expect() on input data; no new crates without approval.
- Every boundary in the specification has a test, predicted before running it.
- cargo test, cargo clippy --all-targets -- -D warnings and cargo fmt --check pass.
- You can explain every line; the decision and evidence are recorded.
- No passwords, keys, personal data or assessment answers were pasted into the AI tool.

Support and Assessment Flow

Assessment attendance, then Written Assessment, then Practical Performance, then upload answers and evidence on the LMS, then the assessor records Competent (C) or Not Yet Competent (NYC).

Course material: <https://lms-tms.tertiaryinfotech.com/>

Support: enquiry@tertiaryinfotech.com / +65 6100 0613 / <https://www.tertiarycourses.com.sg/wsqaivibe-coding-with-rust.html>. Repository: <https://github.com/tertiarycourses/TGS-2023039924-AI-Vibe-Coding-with-Rust>

Sources

The Rust Programming Language (the Book): <https://doc.rust-lang.org/book/>

Rust by Example: <https://doc.rust-lang.org/rust-by-example/>

The Cargo Book: <https://doc.rust-lang.org/cargo/>

The rustdoc Book (documentation tests):

<https://doc.rust-lang.org/rustdoc/write-documentation/documentation-tests.html>

Rust standard library API reference: <https://doc.rust-lang.org/std/>

Install Rust with rustup: <https://www.rust-lang.org/tools/install>

rust-analyzer for Visual Studio Code: <https://rust-analyzer.github.io/>

Clippy lint documentation: <https://doc.rust-lang.org/clippy/>

Rust API Guidelines (documentation and naming): <https://rust-lang.github.io/api-guidelines/>

Serde data model and derive: <https://serde.rs/>

crates.io registry (serde, serde_json): <https://crates.io/>

The workbench illustration is a course graphic that reproduces Activity 1 code and its real test result.